



哈爾濱工業大學(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

FlexKV-OS 设计开发文档

FlexKV-OS – 基于 llama.cpp 的端侧大模型推理框架

队伍名称：一起去爬梧桐山

所属赛题：Proj59

项目成员：苏安炫、李思甜

项目导师：夏文、李诗逸

所属高校：哈尔滨工业大学（深圳）

2026 年 08 月

摘要

随着大语言模型（Large Language Model, LLM）逐步由云端数据中心向个人计算机、边缘设备及嵌入式平台迁移，有限物理内存与较低存储带宽逐渐成为制约端侧大模型部署的重要因素。在资源受限环境下，模型权重需要占用大量常驻内存，自回归推理产生的 KV Cache 又会随上下文长度和并发会话持续增长；与此同时，传统 `mmap` 与操作系统 Page Cache 主要依赖缺页和页面回收进行被动调度，难以利用 LLM 逐层执行、MoE 稀疏专家激活以及会话生命周期等结构化语义，容易产生权重重复驻留、频繁 I/O、缓存抖动以及运行时内存竞争等问题。

针对上述问题，本项目基于 `llama.cpp` 设计并实现了一套面向内存受限环境的**模型结构感知运行时内存治理系统**，将模型权重、MoE 专家、KV Cache 及预取资源统一纳入应用层可控的内存管理体系。系统采用“**加载期规划 + 运行期治理**”的两阶段架构：加载阶段由 Memory Planner 根据 cgroup 内存约束、模型结构、权重规模、KV 预留空间及运行时安全余量自动选择内存管理后端并生成初始预算；运行阶段由 Server Memory Governor 持续感知物理内存压力、数据驻留状态、可回收空间和 I/O 行为，动态协调不同资源之间的竞争关系。

针对不同数据对象的访问特征，系统设计了三类结构感知的显式驻留机制。对于 Dense 模型，Dense Flex 利用 Transformer Layer 固定顺序访问特征，以 Layer 为基本管理粒度，通过**环形工作集、平衡张量锁定、关键权重驻留和自适应预取**，结合**异步权重流式加载**，仅保留当前计算窗口及高收益权重驻留，降低重复权重 I/O。对于 MoE 模型，MoE Buffer 以 (Layer, Expert) 为专家工作单元，结合**工作集热度统计、CLG/EAM 专家预测、组级驱逐、压力感知保护和动态预算**，将完整专家参数集合转化为有限大小的动态工作集。对于 KV Cache，系统将逻辑会话状态与物理驻留状态解耦，以 Block/Cell 为基本粒度构建 RELEASE、OFFLOAD、PREFETCH 与 Exact Restore 生命周期，通过**物理驻留预算、代价感知排序、组传输、读/恢复流水线和预缺页**，在保证历史上下文可精确恢复的同时主动释放低价值 KV 物理页。

在此基础上，系统进一步通过 Server Memory Governor 建立 Weight-KV-Prefetch 全局资源协调机制。Governor 利用**压力门控、候选资源排序、异步动作执行器和统一预取预算全局约束** Dense 权重预取、MoE 专家预取以及 KV 恢复产生的瞬时内存与 I/O 开销；同时通过**内存重分配机制**将真实回收的物理内存重新分配给高收益权重驻留、专家工作集和预取任务，形成“**状态观测—资源决策—执行回收—效果反馈—资源再投资**”的闭环内存治理过程。

实验结果表明，该系统能够有效降低 LLM 推理过程中的真实物理内存占用，并提高受限内存环境下的可运行性。关闭冗余 Repack 后，Dense 模型 RSS 由 7.77 GB 降至 4.56 GB，MoE 模型由 5.73 GB 降至 **3.60 GB**；Dense 显式 Residency 机制将每 Token 权重流式读取量由 2684 MiB 降至 **612 MiB**，吞吐由 1.77 tok/s 提升至 **5.18 tok/s**。MoE Budget Planner v2 在 1500 MB 严格内存限制下，使多个固定预算均无法运行的场

景实现稳定推理。

全局容量边界实验中，MoE 的 `weight_only` 与 `combined_auto` 均可将最低可运行内存门槛由基线的 2800 MB 降至 **1300 MB**，并在 7 个压力点中成功运行 6 个。KV Cache Physical Resident Budget Controller 可将约 3.00 GiB 的 KV 物理驻留连续压缩至 **0.247 GiB**，最大降低 91.8%；Exact Restore 中通过 Transfer Group、流水化恢复与 Prefault，将物理恢复时间降低 **20.62%**。综合实验进一步表明，权重驻留优化、专家工作集管理与 KV 生命周期管理能够在统一调度框架下形成互补，在显著压缩物理内存占用的同时维持可接受的推理吞吐。

综上，本项目将操作系统虚拟内存机制与 LLM 模型结构及运行时语义相结合，实现了从操作系统被动调页向**模型感知、压力感知和收益感知的主动运行时内存治理**转变。该系统无需改变 Transformer 计算语义，即可对 Dense 权重、MoE 专家和 KV Cache 实施差异化驻留与统一资源协调，为有限 DRAM 和存储带宽条件下的大语言模型本地化、边缘化部署提供了一种可实现、可验证并具有扩展能力的系统级优化方案。

关键词：大语言模型推理；llama.cpp；运行时内存管理；权重流式加载；MoE 专家缓存；KV Cache；内存调度；边缘计算

表 1: 赛题目标完成情况

目标编号	完成情况	说明
目标 1	全部完成	完成对 LLM 推理过程的访存分析，揭示模型权重的逐层顺序访问特征与 KV Cache 随上下文增长的线性增长规律，并验证 MoE 模型中专家激活的稀疏性与局部性特征。
目标 2	全部完成	实现针对模型权重与 KV Cache 的运行时时内存管理机制，通过按需加载、动态驱逐与 KV Cache 物理页回收，在保证推理正确性的前提下降低运行时物理内存占用。
目标 3	全部完成	设计并实现异步预取调度器，构建独立后台加载线程池，实现计算与 I/O 的重叠，并设计自适应窗口调优机制。
总计	全部完成	完成 LLM 推理内存优化系统的核心逻辑设计与实现，并完成内存节省和推理性能验证。

目录

1	项目介绍	6
1.1	项目背景及意义	6
1.1.1	大语言模型快速发展，但内存资源成为端侧部署关键瓶颈	6
1.1.2	LLM 推理具有高度结构化访存特征，为主动内存管理提供优化空间	6
1.1.3	KV Cache 动态增长，需要运行时生命周期管理机制	7
1.1.4	从局部优化到 Weight-KV 的全局 I/O 资源调度	7
1.2	项目目标与分析	7
1.3	项目实现内容	9
1.4	项目完成情况	11
1.5	项目开发计划	12
1.6	项目分工	13
2	现有研究调研概况	14
2.1	LLM 推理的访存行为特征	14
2.2	推理框架	15
2.2.1	llama.cpp 推理框架	15
2.2.2	vLLM 推理框架	16
2.3	内存受限场景下的优化方案	17
2.3.1	权重 I/O 优化：模型参数的存储与访问	17
2.3.2	KV Cache I/O 优化：运行时状态管理	17
2.4	边缘端部署的特殊挑战	18
2.5	现有方案的不足与本项目切入点	19
3	系统总体设计	20
3.1	设计目标与总体思路	20
3.2	系统设计原则	21
3.3	系统总体架构	22
3.3.1	推理执行层	22
3.3.2	全局控制与调度层	22
3.3.3	专用内存管理层	23
3.3.4	底层虚拟内存与存储层	23
3.3.5	系统整体运行流程	24
3.4	系统核心组成	24
3.4.1	Dense Flex：基于 Layer 的权重流式管理	25
3.4.2	MoE Buffer：基于 ExpertGroup 的专家缓存管理	25
3.4.3	KV Cache 管理模块	26

3.4.4	统一调度与执行机制	27
3.5	系统设计总结	28
4	模块设计与实现	28
4.1	加载期与推理期内存规划及后端选择	29
4.1.1	加载期自动后端选择	29
4.1.2	Dense Planner v2	31
4.1.3	MoE Budget Planner v2	33
4.1.4	推理期 CPU Hook: Dense 与 MoE 互斥接入	34
4.2	Dense Flex: Layer 级显式权重驻留	35
4.2.1	Dense Ring 的生命周期	35
4.2.2	Request-Wait-Get-Release 闭环	36
4.2.3	Balanced Locking 与 Pin Policy	37
4.3	MoE Buffer: Expert 级显式驻留	38
4.3.1	Warm Working Set Estimator	38
4.3.2	MoE Predictor: CLG、EAM 与 CCT	39
4.3.3	MoE Group-level Eviction	39
4.3.4	MoE Pressure-Aware Protection	40
4.3.5	MoE Clean Reclaim 与 Dynamic Budget	40
4.4	KV Cache 运行时内存管理	41
4.4.1	Block/Cell 生命周期与 Unified Action	41
4.4.2	Physical Budget View 与 RELEASE-first 控制闭环	42
4.4.3	Cost-aware Hot/Cold Claimant Ranking	43
4.4.4	Exact PREFETCH 与 Graph Gate	44
4.4.5	Transfer Group、Read/Restore Pipeline 与 Prefault	44
4.4.6	Cost-aware Hot/Cold 策略	46
4.5	Server Memory Governor	47
4.5.1	Pressure Gate	47
4.5.2	Auction Select 与候选排序	48
4.5.3	Async Action Executor	48
4.5.4	Unified Prefetch Budget	48
4.5.5	Clean Reclaim	49
4.5.6	Reallocation Credit	49
4.5.7	Observation Marker	49
5	系统测试	50
5.1	测试环境	50
5.2	Dense 模型权重管理模块测试结果	51

5.2.1	Repack 优化与基础内存降低	51
5.2.2	Dense 模型权重驻留机制测试	52
5.2.3	Risk-Aware Lock 避免内存风险边界	53
5.2.4	Dense Pin Policy 对权重访问性能的影响	55
5.2.5	Adaptive Ahead 权重预取调度	57
5.3	MoE 模型权重管理测试结果	59
5.3.1	MoE Buffer 工作集边界分析	59
5.3.2	MoE Budget Planner 自动预算选择	60
5.3.3	Pressure-Aware Full Adaptive 跨内存预算测试	61
5.3.4	MoE 事件级调度诊断	62
5.4	KV Cache 管理模块测试结果	64
5.4.1	Physical Resident Budget 控制曲线	64
5.4.2	RELEASE 与 OFFLOAD 的物理收益归因	65
5.4.3	Exact Restore 执行器消融	67
5.4.4	F16 KV 精度与恢复数据量	68
5.4.5	真实多会话 workload 执行	69
5.5	全局自动调度下的内存容量边界与性能	70
5.5.1	容量边界与可运行性	71
5.5.2	成功运行点的性能对比	72
5.5.3	内存-吞吐权衡分析	73
5.6	系统测试结果总结	74
6	当前系统不足与未来方向	75
6.1	运行时资源决策的自适应能力仍需进一步提升	75
6.2	极端内存约束下仍存在内存占用与推理性能的权衡	75
6.3	系统泛化能力与实验验证范围仍有进一步扩展空间	75
7	大语言模型使用说明	76

1 项目介绍

1.1 项目背景及意义

1.1.1 大语言模型快速发展，但内存资源成为端侧部署关键瓶颈

近年来，大语言模型（Large Language Model, LLM）在自然语言理解、代码生成以及智能交互等领域取得了快速发展。随着模型规模不断扩大，LLM 已经从云端数据中心逐渐向个人计算机、边缘服务器以及嵌入式设备等资源受限环境迁移。相比云端服务器，端侧设备通常具有更有限的物理内存容量、更低的存储访问带宽以及更严格的功耗约束，使得大规模模型的高效部署面临巨大挑战。

传统云端部署方式通常依赖大容量 DRAM、高速显存以及分布式计算资源，将模型参数长期驻留在高速存储中。然而，在内存容量有限的端侧设备中，直接加载全部模型参数往往不可行。以当前主流的 Llama 系列模型为例，即使采用低比特量化技术，数十亿参数规模模型的权重仍然可能达到数 GB 至数十 GB。

现有基于内存映射文件（memory map, mmap）的按需加载方案虽然能够突破物理内存容量限制，但其主要依赖操作系统缺页机制进行被动调度，无法感知 LLM 推理过程中的结构化访问模式，容易产生频繁 I/O 等待以及不可控的 page cache 占用。除静态模型权重外，自回归生成过程中用于保存历史上下文信息的键值缓存（Key-Value Cache, KV Cache）会随着输入长度和生成 Token 数量持续增长，进一步加剧运行时内存压力。

因此，在有限物理内存条件下，如何根据 LLM 推理特点主动管理模型权重、KV Cache 以及运行时数据驻留状态，实现低内存占用与高推理性能之间的平衡，成为当前端侧大模型系统优化领域的重要问题。

1.1.2 LLM 推理具有高度结构化访存特征，为主动内存管理提供优化空间

大语言模型推理过程在计算模式上具有显著的结构化特征：在自回归生成时，模型需依次遍历 Transformer Decoder 的所有层，权重访问严格遵循 $Layer_0 \rightarrow Layer_1 \rightarrow \dots \rightarrow Layer_N$ 的固定顺序，这为从操作系统被动分页向应用层主动内存管理转变提供了前提。

对于 Dense 模型，每一 Token 均完整重复该层级序列，使得权重读取具有高度确定性和可预测的时间局部性。这种层级的顺序访问规律，使得系统能够提前预判未来若干步内的权重需求，从而有机会在数据真正被访问之前主动发起预加载或驻留调整，有效隐藏访存延迟并减少缺页中断。

对于 MoE 模型，虽然模型整体参数规模巨大，但每个 Token 仅依据路由器输出的 Top-K 专家索引激活极小比例的参数。这种“全局稀疏、局部密集”的特征，使得权重驻留策略不必以整个模型为单位，而是可以细化到专家粒度：通过跟踪路由历史或结合请求上下文，推理系统能够预估下一 Token 可能激活的专家集合，并据此动态调整高优先级专家权重在快速存储介质中的驻留比例，同时将低概率专家权重提前换出或延迟加

载，从而在有限内存容量下最大化有效权重命中率。

1.1.3 KV Cache 动态增长，需要运行时生命周期管理机制

大语言模型推理阶段，KV Cache 是另一类关键内存资源，其占用量随上下文长度近似线性增长，在长文本生成、多轮对话及高并发场景下极易膨胀，与静态的模型权重形成激烈竞争，持续挤占有限的物理内存空间。

与权重不同，KV Cache 并非预先加载的静态对象，而是推理过程中动态生成的中间状态，其生命周期依附于特定会话、Token 序列及访问频次，具有明显的“按需产生、随用随存”特征，因而无法采用统一的全局加载/卸载策略。这一差异要求系统必须将 KV Cache 的逻辑存在与其物理驻留状态彻底解耦，转而构建运行时感知的生命周期管理机制。

具体而言，该机制需实时监测各会话的 KV 访问热度与时间局部性，并据此实施动态调度，抑制 KV Cache 随上下文增长而无限膨胀的趋势，将物理内存占用稳定在可控阈值内，避免因长上下文累积导致的内存溢出或频繁交换抖动。最终，该机制在有限资源下显著提升长序列推理的稳定性和并发能力，同时为模型权重驻留保留更充裕的内存预算，使整体资源分配更趋均衡。

1.1.4 从局部优化到 Weight-KV 的全局 I/O 资源调度

现有面向大语言模型推理的内存优化工作往往聚焦于单一资源类型，例如仅优化模型权重的按需加载，或仅对 KV Cache 实施压缩与换出。然而，在实际推理系统中，模型权重、KV Cache、预取缓冲及计算中间结果并非独立存在，它们共同竞争有限的物理内存容量与存储带宽，且相互制约。这类非独立的资源竞争关系决定了任何单一维度的局部优化均无法取得系统整体最优，亟需从全局视角重新设计资源协同策略。

针对上述挑战，本文提出一种面向 LLM 推理的运行时内存治理框架。该框架不预设固定资源配额，而是依据当前推理阶段、模型结构特征以及实时内存压力，动态协调各类资源的驻留比例，并通过实时采集各模块的内存占用、访问延迟和命中率，动态调整各级缓存大小及预取深度，避免局部策略产生的冲突。

通过将模型权重、KV Cache 和计算中间结果等数据统一抽象为可管理的资源实体，并通过系统级调度机制实现多目标协同，系统实现了从“数据访问预测”到“资源驻留控制”再到“压力感知回收”的完整闭环，使得大语言模型能够在受限物理内存环境下稳定运行，显著提升长上下文、多会话及混合专家场景下的吞吐与响应性能。

1.2 项目目标与分析

针对大语言模型在资源受限环境下推理时面临的内存容量不足、I/O 延迟显著以及运行时资源竞争等问题，本项目结合 LLM 推理过程固有的结构化访存规律，引入操作系统虚拟内存管理思想，设计面向推理全过程的运行时内存治理框架。该框架旨在突破

物理内存容量限制，在有限资源下实现稳定、高效的推理服务，具体围绕以下四个层面展开。

1. 分析 LLM 推理过程中的访存行为特征

首先基于 llama.cpp 的执行流程，对模型权重、KV Cache 及运行时缓冲区的访问规律进行系统分析：

- 针对 Dense 模型，重点考察 Decoder Layer 顺序执行中权重的访问顺序、层间数据复用关系以及不同驻留策略对 I/O 行为的影响；
- 针对 MoE 模型，聚焦 Router 激活结果导致的专家稀疏访问特征，研究专家权重的访问频率、时间局部性及动态工作集变化规律；
- 对于 KV Cache，分析 KV Cache 随上下文长度增长的内存占用变化，以及多会话场景下不同 KV 状态的生命周期差异。

通过上述分析，将隐式的访存规律转化为可被系统感知和调度的显式信息，为后续主动管理策略提供数据基础。

2. 利用显式驻留管理机制降低运行时物理内存占用

传统 mmap 方案依赖操作系统缺页中断和 page cache 完成数据管理，应用层无法精确控制模型权重及 KV Cache 的实际驻留范围。本项目基于 LLM 结构化访问特点，将权重和 KV Cache 转化为应用层可管理对象，并针对不同数据类型实施差异化的驻留控制策略：

- 面向 Dense 模型，采用 Layer-level 权重流式管理，仅保留当前计算窗口内必要的若干层权重，随执行进度动态切换驻留集合；
- 面向 MoE 模型，实施 Expert-level 工作集管理，根据路由预测和历史访问统计，缓存实际激活或具有高复用价值的专家参数；
- 对于 KV Cache，根据会话活跃程度和访问热度，区分驻留（Resident）、释放（Release）与恢复（Restore）三种状态，按需调整物理占用。

通过上述动态驻留控制，使推理过程能够在远小于完整参数规模的物理内存中完成，有效突破容量瓶颈。

3. 利用预测预取机制隐藏数据加载延迟

在显式限制工作集后，部分数据需要在推理过程中动态加载。若采用同步读取方式，每次缺失均会造成推理线程阻塞。由于 LLM 推理过程具有高度可预测性，本项目设计异步预取机制，实现计算与 I/O 的重叠：

- Dense 模型依据当前 Layer 执行位置，提前预取后续若干层权重，将层级顺序访问的确定性转化为预取窗口；

- MoE 模型根据专家访问历史与 Router 输出预测，提前准备未来可能激活的 Top- K 专家参数，降低稀疏访问带来的加载抖动；
- 在 KV Cache 恢复场景中，利用上下文窗口移动的规律，提前加载即将需要的历史状态，掩盖恢复延迟。

预取操作由独立的后台线程异步执行，与推理流水线并行，从而有效降低存储访问对 Token 生成速度的影响。

4. 构建 Weight-KV-I/O 全局资源协调机制

模型权重、KV Cache 和预取数据共同竞争有限的物理内存资源，单一模块的独立优化难以取得系统整体最优。为此，本项目进一步设计全局资源协调器（Memory Governor），将 Dense 权重、MoE 专家权重、KV Cache 以及预取缓冲区统一纳入运行时管理体系。该协调器综合考量以下因素：

- 当前物理内存压力和剩余可用容量；
- 各模块当前驻留状态及可回收空间；
- 各类数据的历史访问频次与预期访问收益；
- 存储带宽占用情况及 I/O 竞争程度。

基于上述输入，Memory Governor 动态决策各类数据的驻留、回收与预取优先级，并在不同模块间重新分配内存预算。通过全局视角的协同调度，系统从传统局部缓存优化演进为面向 LLM 推理全过程的闭环内存治理框架，在保证服务质量的前提下最大化有限内存资源的利用率。

1.3 项目实施内容

针对上述目标，本项目围绕 LLM 推理过程中的权重管理、KV Cache 管理以及全局资源调度三个方向展开系统设计与实现。整体实现划分为四个主要部分：基础环境与访存特征分析、分层资源管理机制（涵盖 Dense 权重、MoE 专家和 KV Cache）、全局协同调度与资源再分配，以及系统集成与验证。各部分内容如下：

1. 基础环境搭建与访存特征分析

首先在 Ubuntu 22.04/24.04 环境下搭建基于 llama.cpp 的开发与测试平台，完成模型加载流程分析、源码修改及性能测试工具配置。项目使用 GCC、CMake 等工具编译推理框架，并结合 cgroup v2、taskset、perf 及 RSS/内存监控接口构建资源受限实验环境，以模拟端侧设备中的内存约束场景。

在此基础上，基于 llama.cpp 源码与 GGUF 模型文件结构，对模型参数布局、张量组织方式及运行时访问路径进行系统分析。重点考察 Dense 模型的 Layer 顺序

访问规律、MoE 模型的 Expert 稀疏激活特征、mmap/page cache 的驻留行为、权重加载与计算阶段的时序关系，以及 KV Cache 随上下文长度增长的内存占用趋势。通过日志埋点和运行时统计，建立 LLM 推理过程的访存行为模型，为后续主动管理提供数据驱动基础。

2. 分层资源管理机制：权重与 KV Cache 的差异化控制

针对不同数据类型，本项目设计了三套独立但协同的运行时管理模块，分别应对 Dense 权重、MoE 专家权重和 KV Cache 的资源治理需求：

(1) Dense Flex 权重管理模块：针对 Dense 模型所有层权重均需访问的特点，实现层级权重流式管理。核心机制包括按执行进度滑动工作窗口、动态调整窗口大小和平衡锁定等。通过控制层级权重工作窗口大小，系统仅保持当前推理阶段高价值权重驻留，显著降低 Dense 模型运行时的峰值内存。

(2) MoE Buffer 专家工作集管理模块：针对 MoE 模型专家规模庞大但单 Token 激活稀疏的问题，实现专家级缓存管理。系统以单个 Expert 为基本管理粒度，构建专家组（Expert Group）工作单元，并设计专家切片流式加载、热点专家保护、以及动态专家预算调整等机制。通过仅缓存高概率访问的专家，将完整专家集合转化为有限大小的动态工作集，在保持较高命中率的同时大幅压缩内存占用。

(3) KV Cache 运行时管理模块：针对长上下文推理中 KV Cache 持续增长的问题，实现运行时生命周期管理。该模块持续监测 KV 块的驻留状态与可回收状态，根据系统内存压力动态执行释放策略、序列级换出以及会话恢复机制，并通过 KV Cache 软预算约束控制整体内存占用上限。通过将逻辑会话状态与物理驻留解耦，低频访问的 KV 数据可及时释放物理内存，为其他模块腾出空间。

3. 全局协同调度与资源再分配闭环。

为避免各模块独立优化导致的资源冲突，本项目设计统一的 Server Memory Governor，统筹协调 Dense 权重、MoE 专家及 KV Cache 之间的内存竞争。Governor 周期性获取系统内存压力、各模块当前驻留状态及可回收空间，并根据优化目标生成对应控制策略，包括数据预取、缓存收缩、资源回收等操作，实现全局资源的最优分配。

在此基础上，进一步设计 Unified Prefetch Budget 机制，防止 Dense、MoE 和 KV 恢复过程同时触发大量预取请求造成瞬时内存尖峰。因此，Governor 根据当前系统可用内存空间、内存压力状态以及存储访问负载，动态调整各模块的预取预算，在隐藏 I/O 延迟和控制额外资源开销之间取得平衡，提升系统稳定性。

此外，项目引入 Reallocation Credit 资源再分配机制，形成“释放—收益—再投资”的闭环。当系统执行权重清理回收、缓存预算收缩或 KV Cache 释放等操作后，系统根据实际释放效果生成内存资源额度，并允许在安全内存范围内将这些额

度重新投资于高收益权重驻留、专家工作集扩大或关键数据预取能力提升，从而实现资源的动态优化循环利用。

4. 系统集成与实验验证

将上述所有模块统一集成至 `llama.cpp` 推理流程，完成模型加载测试、内存压力测试、RSS 变化分析、KV 长上下文测试、Weight-KV 组合测试以及多模型兼容性测试。通过对比实验，验证系统在有限物理内存条件下能否有效降低运行时 RSS、维持稳定推理吞吐，并避免因内存不足导致的 OOM 或频繁交换抖动。最终评估整体框架在端侧资源受限场景下的实用性与性能增益。

1.4 项目完成情况

在决赛阶段，本项目已经完成面向资源受限环境的大语言模型运行时内存治理系统设计与核心功能实现。系统基于 `llama.cpp` 推理框架，将模型权重、KV Cache 以及预取数据统一纳入应用层可控的资源管理体系，实现了从单点权重优化向 Weight-KV-I/O 全局协同优化的扩展。

目前系统已经完成以下核心模块：

- Dense Flex：面向 Dense 模型的 Layer-level 权重流式管理；
- MoE Buffer：面向 MoE 模型的 Expert-level 工作集管理；
- KV Cache Manager：面向长上下文场景的 KV 生命周期管理；
- Memory Planner：基于内存约束自动生成初始资源配置；
- Server Memory Governor：面向全局资源竞争的运行时调度框架；
- Unified Prefetch Budget：跨模块预取资源协调机制。

系统已完成端到端集成，并在不同模型规模和内存限制条件下开展测试验证。

表 2: 项目核心功能完成情况

实现内容	对应目标	完成情况	说明
基础环境搭建与访存特征分析	目标 1	完成	完成 <code>llama.cpp</code> 编译部署与 <code>cgroup v2</code> 资源受限环境搭建，建立访存行为分析模型。

实现内容	对应目标	完成情况	说明
加载期 Memory Planner	目标 2、4	完成	根据系统内存大小、模型规模与结构类型，自动生成初始资源配置，减少人工调参依赖。
Dense Flex 权重管理模块	目标 2、3	完成	面向 Dense 模型实现 Layer-level 权重流式管理，有效控制常驻内存集大小。
MoE Buffer 专家管理模块	目标 2、3	完成	以单个专家为粒度构建 Expert-Group 工作单元，将全局专家集转化为有限动态工作集。
KV Cache 运行时管理模块	目标 1、2	完成	实现 KV 块常驻与可回收状态监测与转换，通过逻辑与物理驻留解耦降低长上下文内存占用。
统一预取与 I/O 重叠调度	目标 3、4	完成 核心框架	根据系统可用内存空间与 I/O 负载动态分配预取额度，实现后台异步预取与推理计算并行执行。
全局资源协调与再分配机制	目标 4	完成 核心框架	实现 Server Memory Governor，周期性采集内存压力及各模块状态，将释放资源转化为可再投资的信用，形成释放—收益—再分配闭环。
系统集成与验证	目标 1、2、3、4	完成验证	完成各模块在 llama.cpp 推理流程中的统一集成，开展 RSS 控制、KV 长上下文、Weight-KV 组合及多模型兼容性测试，验证有限内存条件下的推理稳定性。

因此，本项目当前已经完成赛题要求的核心功能，并进一步形成面向未来端侧大模型部署的全局运行时内存治理框架。

1.5 项目开发计划

表 3: 项目开发计划

阶段内容	时间安排
LLM 推理机制调研、llama.cpp 源码分析、GGUF 文件结构研究以及实验环境搭建	第 1-2 周
完成基础权重流式加载机制，实现 Dense Layer Streaming、MoE Expert Buffer 原型，并完成正确性验证	第 3-5 周
实现 KV Cache 生命周期管理机制，包括 resident 状态分析、释放策略以及长上下文测试	第 5-6 周
实现异步预取机制，完成计算与 I/O 重叠优化，并进行参数调优	第 6-7 周
完成 Memory Planner 与 Server Memory Governor 基础架构，实现 Weight-KV-Prefetch 统一资源管理	第 8 周
完善 Unified Prefetch Budget、Dynamic Budget、Reallocation Credit 等全局优化机制	第 9 周
开展 Dense、MoE、KV 以及 Memory Governor 独立消融实验，分析各模块贡献	第 10-11 周
针对不同模型规模、不同 memory cap 和不同存储条件开展稳定性测试	第 12 周
进一步优化 I/O 路径、缓存策略以及资源调度策略，扩展系统对于长上下文、多会话场景的适应能力	第 13-14 周

1.6 项目分工

本项目围绕“模型权重主动管理”和“运行时内存资源治理”两条技术路线展开，并共同完成全局 Memory Governor 的系统集成。

表 4: 项目分工

小组成员	分工内容
苏安炫	负责 LLM 权重侧优化相关工作，包括 llama.cpp 框架与 GGUF 模型格式分析，Dense Flex 权重流式管理、MoE Buffer 专家缓存机制、权重预取调度以及相关性能测试。同时负责 Memory Planner 中权重侧预算规划接口设计。

小组成员	分工内容
李思甜	负责 KV Cache 与运行时内存管理相关工作，包括 LLM 推理访存分析、KV Cache 生命周期管理、压力检测、释放恢复机制以及长上下文压力测试。同时负责 Memory Governor 中 KV 资源状态接口设计。
共同负责	负责系统总体架构设计、Server Memory Governor 集成、Weight-KV-Prefetch 全局资源协调机制设计、实验环境搭建、Benchmark 测试、结果分析以及参赛文档和答辩材料整理。

2 现有研究调研概况

本节主要介绍和项目有关的大语言模型推理技术、内存管理方案以及边缘部署优化方案的研究情况，并根据调研结果寻找本项目的切入点。

2.1 LLM 推理的访存行为特征

LLM 推理过程具有独特的访存行为特征，这是本项目所有优化设计的出发点。LLM 推理通常分为 Prefill 阶段和 Decode 阶段。

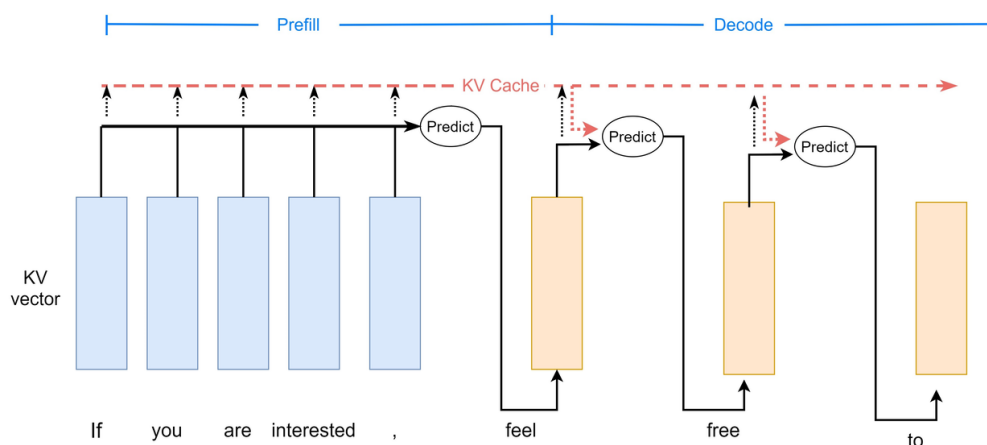


图 1: LLM 推理中的 Prefill 与 Decode 阶段

Prefill 阶段并行处理全部输入 Token，在每个注意力层为每个 Token 计算 Key 和 Value 向量，通常具有较高并行度。Decode 阶段以自回归方式逐 Token 生成，每个新 Token 需要对所有先前 Token 的 Key-Value 对进行注意力计算，因而更容易受内存带宽限制。

在访存行为方面，LLM 推理过程中的数据搬运可分为三个独立的 I/O 流：

1. 权重 I/O

每解码一步需要加载模型权重。对于大规模模型而言，权重 I/O 是推理过程中最主要的数据搬运来源之一。权重 I/O 与批次大小无关，可通过批处理进行摊销。

2. KV Cache I/O

每层每序列需要读取历史 Token 的键值缓存。KV Cache I/O 随序列长度和批次大小线性增长，在长上下文场景下可能接近甚至超过模型权重本身。

3. 激活 I/O

激活 I/O 主要来自中间激活张量在 kernel 启动间的读写。Decode 阶段的激活 I/O 通常远小于权重 I/O 和 KV Cache I/O；Prefill 阶段若不使用 FlashAttention，则可能产生较高的中间读写开销。

上述分析表明，权重 I/O 和 KV Cache I/O 是 Decode 阶段的两大主要瓶颈。两者之间存在动态平衡关系：权重 I/O 可通过批次处理摊销，但 KV Cache I/O 随批次和上下文长度线性增长。这一关系是后续优化方案所面临的核心挑战，也是本项目统一管理权重与 KV Cache 的动机所在。

2.2 推理框架

2.2.1 llama.cpp 推理框架

llama.cpp 是一个纯 C/C++ 实现的大语言模型推理引擎，旨在为广泛的硬件设备提供推理能力，从智能手机等边缘设备到多 GPU 服务器集群均可覆盖。其具有如下特点：

1. 纯 C/C++ 实现，零依赖，无需引入 PyTorch 等深度学习框架。
2. 量化作为一等公民，原生内置多种量化格式。
3. 支持 CPU 与 GPU 混合推理，当模型大于显存时可自动拆分至 CPU 内存和 GPU 显存。

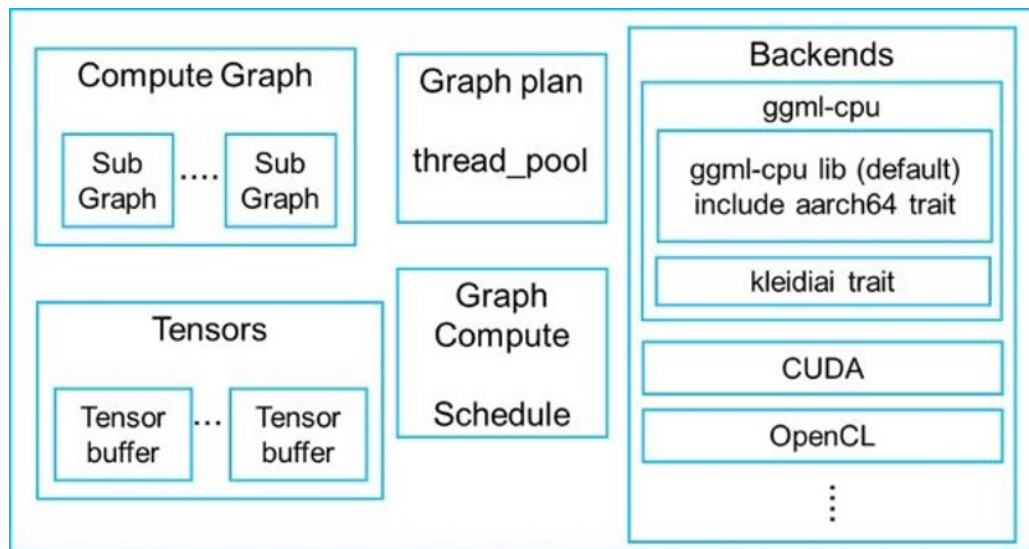


图 2: llama.cpp 推理框架架构

llama.cpp 采用分层架构设计：GGML 层提供底层张量计算、内存管理和后端抽象；llama 核心层实现模型加载、分词、KV Cache 管理、计算图构建及采样功能；工具层提供 CLI、HTTP server 和基准测试工具等。

llama.cpp 当前通过 mmap 机制实现模型参数按需加载。当推理线程访问某层参数时，操作系统通过缺页中断从存储设备加载相应数据页。该机制实现简单，但在物理内存不足时，几乎每次权重访问都会触发 I/O，导致推理吞吐显著下降。本项目使用 llama.cpp 作为基础框架进行二次开发，在其数据加载路径中插入滑动窗口、异步预取、权重流式替换和 KV Cache 回收逻辑。

2.2.2 vLLM 推理框架

vLLM 是一个面向数据中心的高吞吐量 LLM 推理框架，其主要贡献是提出了 PagedAttention 机制。

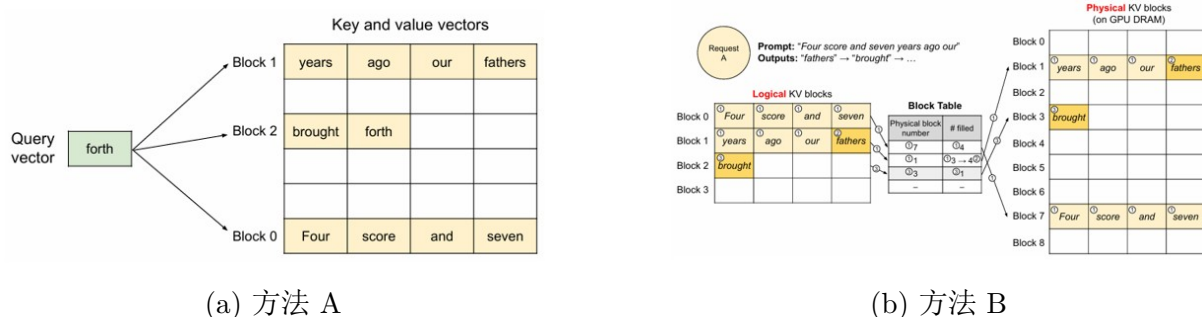


图 3: PagedAttention 机制示意图

PagedAttention 将 KV Cache 切分为固定大小的页或 block，按需分配，其思想与操作系统虚拟内存页表相似。一个 block table 将逻辑页映射到物理内存，使 KV Cache

可非连续存储。PagedAttention 还支持前缀缓存，即多个请求共享同一前缀时，对应的 KV Cache 页可直接复用而非重新计算。vLLM 主要面向 GPU 服务器环境，但其将 KV Cache 视为可灵活管理内存资源的思想，为本项目在边缘端设计 KV Cache 管理策略提供了参考。

2.3 内存受限场景下的优化方案

由前文访存行为分析可知，内存压力主要来源于两类对象：模型权重与运行时 KV 缓存。前者对应参数存储与传输成本，后者对应生成过程中持续增长的内存消耗。因此，内存优化可以从权重 I/O 与 KV cache I/O 两个方向进行探索。

2.3.1 权重 I/O 优化：模型参数的存储与访问

权重开销主要由模型规模决定，其核心问题在于参数无法完全驻留在计算设备中，必须在不同存储层之间迁移。当前主流的方向有：

量化压缩 量化通过降低参数数值精度来减少存储体积，从而降低加载与传输开销。常见做法是将浮点权重映射到低比特整数表示，并在推理阶段进行近似计算。该类方法的特点是压缩比例在部署阶段基本固定，一旦确定量化策略，模型的内存占用也随之固定，因此难以根据实时资源情况进行调整。

分层存储与按需加载 另一类方法通过构建多级存储结构，将权重分布在内存与外部存储介质上，并在计算时按需调入。典型实现包括基于操作系统的惰性加载机制，以及基于用户态控制的主动预取策略。前者依赖访问触发数据加载，行为不可控；后者通过预测未来访问路径提前调度权重，从而减少计算等待时间。

2.3.2 KV Cache I/O 优化：运行时状态管理

KV cache 是自回归推理过程中逐步增长的中间状态，其规模随生成长度线性增加，是动态内存压力的主要来源。现有的优化方案有：

分页式管理 通过将缓存划分为固定大小的块，并使用映射结构管理逻辑与物理地址关系，可以避免连续内存分配带来的碎片问题，并支持多个请求的并发执行。这种方式将缓存管理从连续内存模型转变为块级资源调度问题，使得系统能够在有限内存条件下支持更大规模的并发负载。

缓存回收与迁移 在内存不足或负载变化时，可以将部分缓存数据从高速存储迁移至低速存储，或直接释放不再需要的部分内容。驱逐策略通常基于优先级或重要性估计，例如根据访问频率、位置特征或注意力强度进行判断。

缓存压缩 另一类方法通过降低缓存中数值表示的精度来减少存储占用，但由于缓存状态依赖生成历史，其误差会在后续计算中逐步传播，因此需要在压缩率与质量之间进行权衡。

结构性压缩 从模型结构层面减少缓存规模的方法包括减少注意力头数量、共享键值表示，或使用低维隐变量替代显式缓存表示。这类方法可以从源头降低缓存生成量，而不是在生成后进行压缩处理。

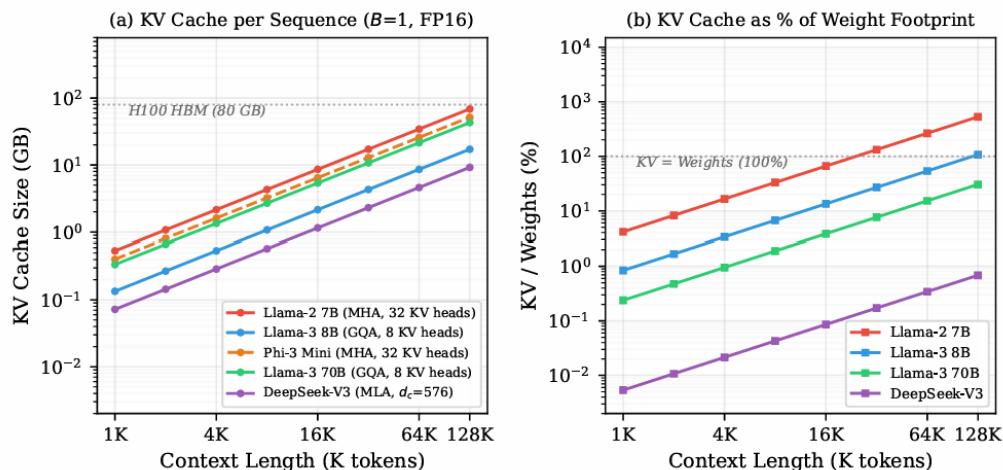


图 4: 架构级 KV 缩减机制

2.4 边缘端部署的特殊挑战

边缘端设备部署 LLM 面临与主流部署环境（如云端、数据中心、服务器集群等）不同的资源约束，其主要挑战体现在以下几个方面：

1. 内存容量严格受限

边缘设备 DRAM 容量通常较小，难以容纳大规模模型权重，即使采用 4-bit 量化仍可能无法完整加载。

2. 存储带宽有限

边缘设备多采用 eMMC 或 UFS 闪存，其顺序读写带宽显著低于主流部署环境中的高性能存储系统，导致模型权重加载与 KV Cache 访问的 I/O 延迟更难被隐藏。

3. 计算能力受限或异构性不足

部分边缘设备仅依赖 CPU 进行推理，或配备的 GPU 计算能力与显存规模有限，难以支撑大规模模型的高效执行。

4. 功耗与散热约束

边缘设备受电池容量与散热条件限制，无法持续维持高功耗计算状态。

上述挑战表明，面向主流部署环境的优化方法难以直接迁移至边缘端，需要针对其存储层次结构与计算资源约束设计专门的系统级优化方案。

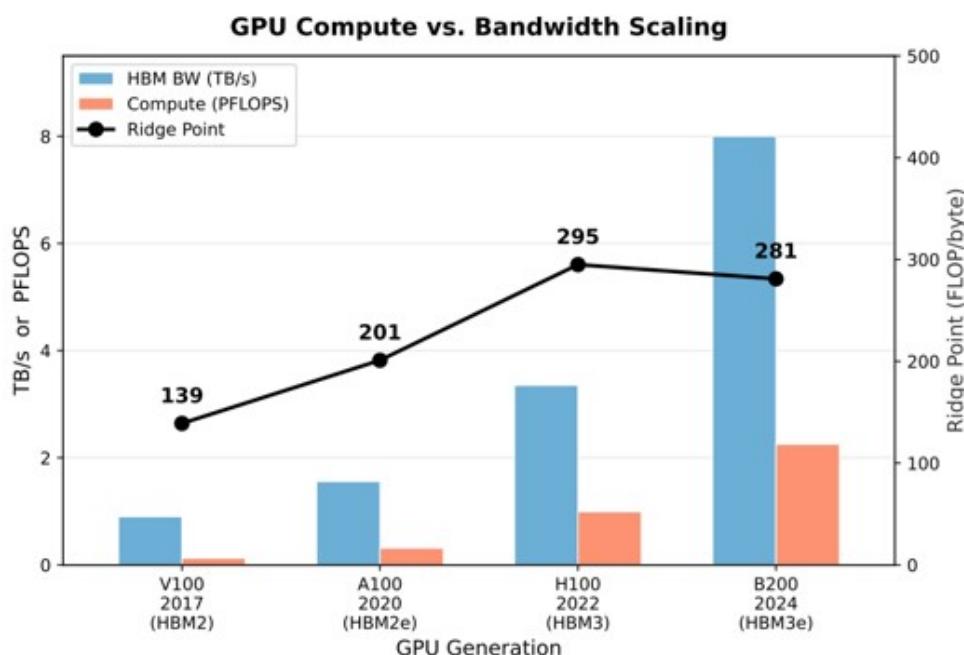


图 5: 边缘端大模型推理资源限制

2.5 现有方案的不足与本项目切入点

综合调研可知，现有方案在内存受限环境下的 LLM 推理中存在以下不足：

1. 权重与 KV Cache 独立管理，未形成统一视图

llama.cpp 通过 mmap 管理权重，通过连续分配管理 KV Cache，两者互不协调。当 KV Cache 增长时，可能挤占权重缓存空间。权重与 KV Cache 在时间维度上存在明确对应关系，但这一关系尚未被充分利用。

2. 权重管理缺乏访存模式感知

现有方案的加载与驱逐策略未充分利用 LLM 推理逐层顺序访问的确定性和可预测性。mmap 方案是被动加载，缺少主动预测；已有预取方案虽具主动性，但未建立显式的层级访问预测模型。

3. KV Cache 管理未适配边缘端 I/O 层次

现有 KV Cache 管理方案多面向 GPU 环境设计，未充分考虑边缘端 DRAM 到 SSD 的 I/O 特性。当 KV Cache 换出至 SSD 时，如何预取、何时换入、如何与权重加载共享 I/O 带宽，仍缺乏系统级方案。

上述问题的核心在于，现有方法通常将权重 I/O 与 KV Cache I/O 作为两个独立优化目标分别处理，而未考虑二者在推理过程中对内存容量与 I/O 带宽的共同竞争关系。这导致单点优化难以从系统层面消除瓶颈。

Configuration	W (GB)	K (GB)	Total (GB)	Dom. Flow	Est. TPOT (ms)
FP16 baseline	141	42	183	W	68
+ INT4 weights (AWQ)	35	42	77	K	29
+ INT4 KV cache (KIVI)	35	10.5	45.5	W	17
+ 2:4 sparsity	17.5	10.5	28	W	10.4
+ Spec. decoding ($\gamma=5, \alpha=0.8$)	4.4*	10.5	14.9	K	5.6

图 6: 权重 I/O 与 KV Cache I/O 孤立优化的瓶颈效应

基于上述分析，本项目构建统一的运行时内存管理框架，将模型权重与 KV Cache 纳入同一调度体系。在该框架下，以层、MoE 专家以及 KV block 为基本管理粒度，对权重驻留状态与 KV Cache 状态进行联合管理；结合 CLG 预测机制实现跨层预取调度，并通过异步加载与驱逐策略实现 I/O 与计算的重叠执行。当权重或 KV block 进入非活跃状态时，系统将其纳入可回收集合，并在资源压力或预测触发下执行换入与换出操作，从而在内存受限环境下实现更稳定的推理执行与更高的资源利用效率。

3 系统总体设计

3.1 设计目标与总体思路

在资源受限环境中，大语言模型推理面临的核心问题并不是单纯降低模型大小，而是在有限物理内存条件下合理决策哪些模型权重需要长期驻留、哪些数据可以按需加载、哪些缓存可以暂时释放、哪些数据值得提前预取，以及回收后的资源应该重新分配给哪个对象。

传统基于 mmap 和操作系统 page cache 的方式能够支持模型运行，但其数据驻留决策主要由操作系统根据缺页访问和页面回收机制完成，无法感知 LLM 推理过程中的高级语义信息——例如当前正在执行的 Transformer Layer、下一阶段可能访问的权重区域、MoE Router 选择的 Expert 集合，以及 KV Cache 当前所属 Session 的生命周期状态。

基于上述分析，本项目的总体设计目标是：利用 LLM 推理过程中的结构化访存特征，在用户态建立面向模型语义的运行时内存治理层，使系统能够主动管理数据驻留状态，而不是完全依赖操作系统被动调页。围绕这一目标，系统采用“两阶段内存治理”设计：

阶段一：加载期 Memory Planner 在模型加载阶段，系统首先根据当前资源环境和模型结构生成初始内存规划方案。Planner 综合考虑系统可用内存限制、模型权重规模、Dense 或 MoE 模型类型、KV Cache 预留空间以及运行时安全余量，据此选择不同

的管理后端：Dense 模型采用 Layer 级权重流式管理，MoE 模型采用 Expert 级工作集管理，KV Cache 根据运行需求建立动态预算，建立适应当前环境的初始资源配置。

阶段二：运行期 Server Memory Governor 由于真实推理过程中的资源需求具有动态变化特点，仅依靠加载阶段规划无法长期保持最优状态，系统进一步引入运行期 Server Memory Governor。Governor 周期性获取系统内存压力、cgroup 内存状态、权重驻留情况、Expert 工作集状态以及 KV Cache 使用情况，并根据当前状态生成回收、释放、扩容、缩容、预取、重映射等控制动作。通过加载期规划与运行期治理相结合，系统能够在保证推理正确性的基础上，根据实际运行状态动态调整内存资源分配。

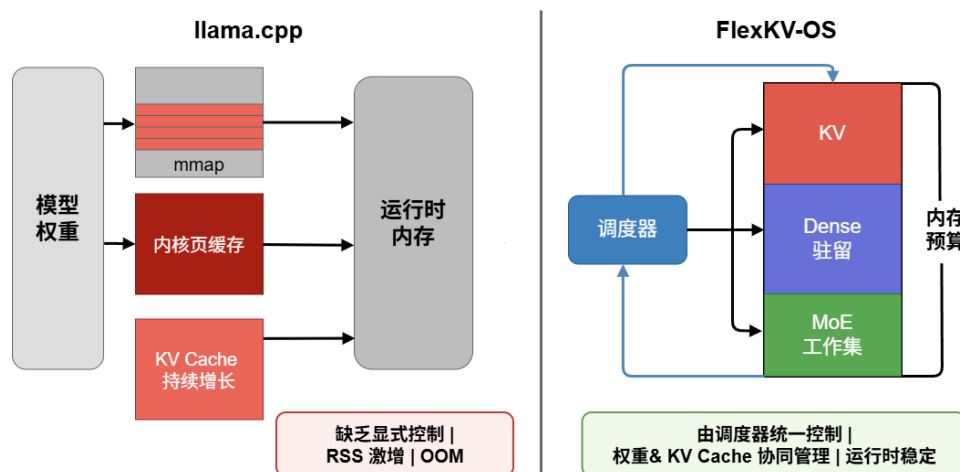


图 7: 系统对比图

3.2 系统设计原则

针对 LLM 推理过程中的访存特点和资源竞争问题，本系统遵循以下设计原则：

1. 模型结构感知的资源管理

不同模型结构具有不同的数据访问模式，因此系统不采用统一缓存策略，而是根据模型结构设计对应的管理机制。通过结构感知设计，使系统能够利用模型自身访问规律提高内存利用效率。

2. 显式驻留优先于被动调页

传统虚拟内存机制主要关注页面访问、缺页处理和页面回收，但无法理解 LLM 推理中的计算语义。因此，本系统将关键数据对象转化为应用层可感知资源，将内存管理从被动调页转变为模型感知的驻留管理，从而减少不可预测的缺页等待。

3. 计算与数据访问解耦

大模型推理性能的重要瓶颈之一是存储访问延迟，系统通过异步加载机制将数据访问过程从推理计算路径中分离，推理线程仅负责发起需求、等待必要数据和执行计算。通过计算与 I/O 重叠，有效降低数据加载等待时间。

4. 全局资源约束优先

由于 Dense 权重、MoE Expert、KV Cache 以及预取缓冲区均共享有限物理内存，局部模块独立优化可能导致资源竞争。因此，本系统引入统一 Memory Governor，根据当前全局内存余量、压力状态、I/O 负载与预期收益决定资源调整方向。

5. 回收与资源再利用闭环

传统内存优化通常只关注内存压力触发回收的单向过程，但在多模块 LLM 推理环境中，仅释放资源并不足以获得最佳收益。本系统进一步引入资源反馈机制，将释放资源重新分配给收益更高的数据对象，从而实现内存资源的动态循环利用。

6. 保持推理语义不变

系统所有优化均作用于数据管理路径，而不修改 Transformer 计算逻辑，在降低运行时内存占用的同时，保证模型推理结果的一致性。

3.3 系统总体架构

本系统采用分层式运行时内存治理架构，整体由推理执行层、全局控制与调度层、专用内存管理层以及底层虚拟内存与存储层组成。其中，全局控制与调度层中的 Server Memory Governor 是系统核心控制模块，负责感知运行时资源状态，并协调 Dense 权重、MoE 专家权重以及 KV Cache 三类数据对象之间的资源竞争。系统整体数据流和控制流遵循 *Inference* $\rightarrow$ *Memory Observation* $\rightarrow$ *Governor Decision* $\rightarrow$ *Backend Action* $\rightarrow$ *Storage/Memory* 的闭环路径，各层职责与协作关系如下：

3.3.1 推理执行层

推理执行层位于系统最上层，负责完成大语言模型前向计算流程，包括 Transformer Layer 计算、Attention 计算、MoE Router 路由、Expert 计算以及 Token 生成等核心任务。该层保持原始推理框架的计算逻辑完全不变，不直接负责数据生命周期管理，而是通过运行时接口向调度层提供计算过程中的状态信息，包括当前执行 Layer、当前 Token 位置、当前激活 Expert 集合、当前 Session 状态以及 KV Cache 访问需求。这些信息作为 Memory Governor 进行资源决策的重要依据。在 Dense 模型推理过程中，执行层按照固定 Layer 顺序发起权重访问 ($Layer_0 \rightarrow Layer_1 \rightarrow \dots \rightarrow Layer_N$)；而在 MoE 模型中，则根据 Router 输出动态确定专家访问集合 ($Router \rightarrow Top-K Expert$)。因此，推理执行层能够为上层调度提供模型结构相关的访问信息。

3.3.2 全局控制与调度层

全局控制与调度层是系统核心部分，由 Server Memory Governor、Memory Planner、Prefetch Scheduler 和 Resource Monitor 共同组成，负责将模型访问信息转换为具体资源管理动作，实现从应用层请求到底层内存操作的映射。

Memory Planner 负责加载阶段的初始资源规划。该模块根据模型类型、模型规模、系统内存限制、KV Cache 预留空间以及安全运行余量，生成初始配置，包括 Dense Layer Ring 大小、MoE Expert Buffer 容量、KV Cache 基础预算以及预取初始范围，为后续运行时调度建立合理的资源基线。

Server Memory Governor 是运行阶段动态资源管理的核心。该模块周期性获取系统 RSS、cgroup memory 状态、当前数据驻留情况、可回收空间以及 I/O 压力，并根据当前状态生成对应的控制动作：回收低价值数据、释放暂时不需要资源、扩大高收益缓存、降低资源占用、提前加载数据以及调整长期驻留集合。通过该机制，实现 Dense、MoE 和 KV Cache 三类资源之间的动态协调。

Prefetch Scheduler 根据模型访问规律生成未来数据需求预测。对于 Dense 模型，依据当前执行 Layer 预测后续 Layer 权重需求 (*Current Layer* $\rightarrow$ *Future Layers*)；对于 MoE 模型，根据 Router 预测结果生成候选专家集合 (*Router Prediction* $\rightarrow$ *Expert Candidate*)；对于 KV Cache，则根据 Session 恢复状态提前准备需要恢复的 KV Block。所有预取行为均受到统一预算约束，避免过度加载造成额外内存压力。

3.3.3 专用内存管理层

专用内存管理层根据不同数据类型设计不同的管理后端，实现结构感知的数据驻留控制，包含三个核心模块。

Dense Flex 面向 Dense 模型权重管理。由于 Dense 模型具有固定 Layer 顺序访问特点，系统以 Layer 为基本管理粒度，负责 Layer Working Set 管理、权重流式加载、Layer Prefetch、Layer Reclaim 以及驻留集合动态调整。其目标是在保证计算连续性的同时，仅保持当前计算窗口内必要权重驻留。

MoE Buffer 面向专家权重管理。由于 MoE 模型总参数规模巨大但单 Token 仅激活少量专家，系统以 (*layer, expert*) 作为主要管理对象，负责 ExpertGroup 管理、Expert Working Set、Expert Admission、Expert Replacement 以及热点专家保护。通过动态维护有限专家集合，将完整专家空间转化为可控缓存空间。

KV Cache Backend 负责管理推理过程中的动态 KV 状态。系统不直接删除逻辑上的历史 KV，而是区分逻辑 KV 与物理驻留——逻辑上仍属于 Session 的 KV 状态，不一定长期占用物理内存。主要状态包括：RESIDENT（当前驻留内存）、RECLAIMABLE（可安全释放）、SWAPPED（已迁移至外部存储）以及 RESUME_PENDING（等待恢复）。通过状态转换，实现长上下文场景下 KV Cache 的动态生命周期管理。

3.3.4 底层虚拟内存与存储层

底层虚拟内存与存储层由操作系统提供基础资源管理能力，包括 Linux Virtual Memory、mmap 映射机制、Page Cache、DRAM 以及 SSD 或外部存储。系统并不替代操作系统虚拟内存机制，而是在其之上增加模型感知的控制能力，通过 mmap、madvise、

O_DIRECT、pread 以及显式 buffer 管理等接口，实现对数据加载、释放和迁移过程的精细控制。

3.3.5 系统整体运行流程

系统运行过程可以概括为以下步骤：推理执行层开始执行当前 Token 计算，并产生 Layer、Expert 或 KV Cache 访问需求；Memory Governor 根据当前访问信息和系统状态进行资源分析；对于即将访问的数据，若已驻留则直接执行计算，若未驻留则触发对应 Backend 执行加载操作；Backend 根据调度策略执行权重加载、Expert 加载、KV 恢复或数据回收；完成计算后，系统根据反馈信息更新资源状态，并调整下一阶段资源预算。最终形成 *Compute* → *Observe* → *Decide* → *Execute* → *Feedback* 的闭环运行机制。

3.4 系统核心组成

系统核心组成围绕不同类型数据对象的生命周期管理展开，包括模型权重管理模块、KV Cache 管理模块以及统一资源协调机制。由于 Dense 模型、MoE 模型以及 KV Cache 在访问模式和生命周期方面存在明显差异，系统未采用单一缓存策略，而是根据不同数据特征设计对应的管理后端：Dense 模型权重采用基于 Layer 的流式驻留机制，MoE 模型权重采用基于 ExpertGroup 的动态缓存机制，KV Cache 采用基于 Block 的生命周期管理机制。各模块均向 Server Memory Governor 提供资源状态信息，并接受统一的资源调整策略。

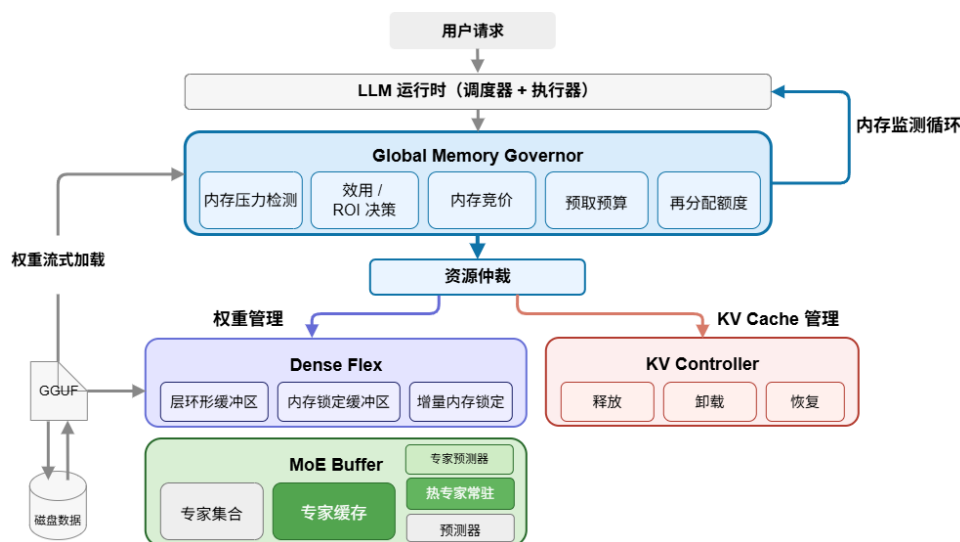


图 8: 主流程图

3.4.1 Dense Flex: 基于 Layer 的权重流式管理

对于 Dense 模型，每个 Token 均需要依次执行全部 Transformer Layer，其权重访问模式为 $Layer_0 \rightarrow Layer_1 \rightarrow \dots \rightarrow Layer_N$ ，具有高度确定性。这意味着系统不需要长期保存全部模型参数，而仅需维护当前计算阶段附近的有效权重集合。Dense Flex 采用 Layer Working Set 管理方式，其核心思想是根据当前执行位置动态维护有限规模的 Layer 驻留窗口，通过提前加载未来访问 Layer 并释放暂时无需求 Layer，降低权重常驻内存。

Layer Working Set 管理根据当前内存预算确定可驻留 Layer 数量，维护当前有效权重集合，主要包括当前正在计算的 Layer、即将访问的预测 Layer 以及根据访问收益长期保留的关键 Layer，其余 Layer 则进入可替换状态。当计算流程移动至新的 Layer 时，系统通过 Layer Streaming 机制根据预测结果提前准备对应权重，数据流路径为 $Storage \rightarrow Weight Buffer \rightarrow Compute$ ，由后台加载线程执行权重迁移，使数据准备过程尽可能与计算重叠。Layer Prefetch 利用 Dense 模型稳定的访问顺序，根据当前 Layer 推测后续访问需求（例如 $Layer_i \Rightarrow Layer_{i+1}, Layer_{i+2}$ ），由后台预取线程提前加载未来 Layer，降低计算线程等待存储访问的概率。当系统检测到内存压力升高时，Layer Reclaim 根据 Layer 最近访问情况、下一次访问距离以及当前驻留收益，选择低收益 Layer 释放，通过动态调整 Layer Working Set，使权重占用能够适应不同内存容量环境。

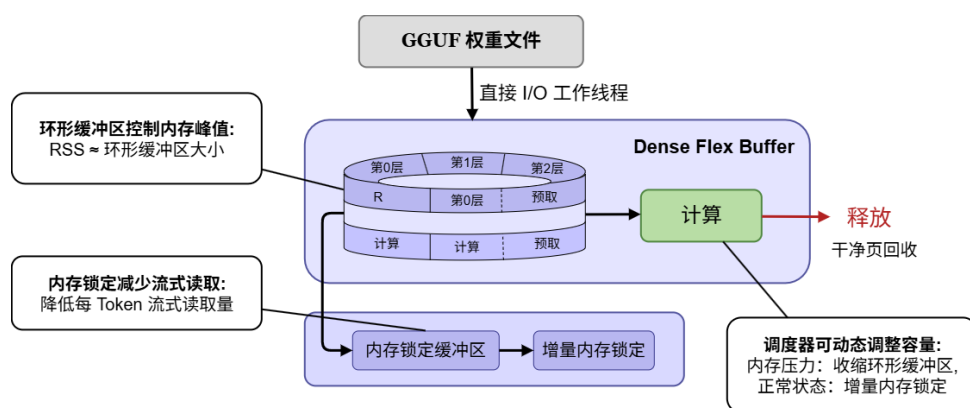


图 9: Dense Flex 模块流程图

3.4.2 MoE Buffer: 基于 ExpertGroup 的专家缓存管理

MoE 模型虽然具有更大的总参数规模，但由于 Router 机制每个 Token 实际仅激活少量 Expert ($Router \rightarrow Top-K Expert$)，因此不适合采用 Dense 模型的 Layer 全量管理方式。本系统设计 MoE Buffer 对专家权重进行独立管理。

系统以 $ExpertGroup = (Layer, Expert)$ 作为基本管理单位，相比以整个模型或整个 Layer 为单位管理，该粒度能够精确描述 Expert 激活情况，支持不同 Layer 专家独立替换，降低无效权重驻留。MoE Buffer 不尝试缓存全部 Expert，而是维护有限大小的动态 Expert 工作集，由当前 Token 激活 Expert、预测可能激活 Expert 以及历史高

频 Expert 三部分组成，其中高频 Expert 作为热点对象优先保留。

当 Router 产生新的 Expert 需求时，Expert Admission 机制判断当前 Expert 是否已驻留、是否存在可用缓存空间以及是否具有足够访问收益，只有满足条件的 Expert 才进入缓存，避免大量低概率 Expert 造成缓存污染。当 MoE Buffer 达到容量限制时，Expert Replacement 综合考虑 Expert 最近访问时间、使用频率、后续预测概率以及当前热点状态，选择低收益 Expert 执行驱逐。通过上述机制，完整专家集合被转换为有限大小的动态工作集。

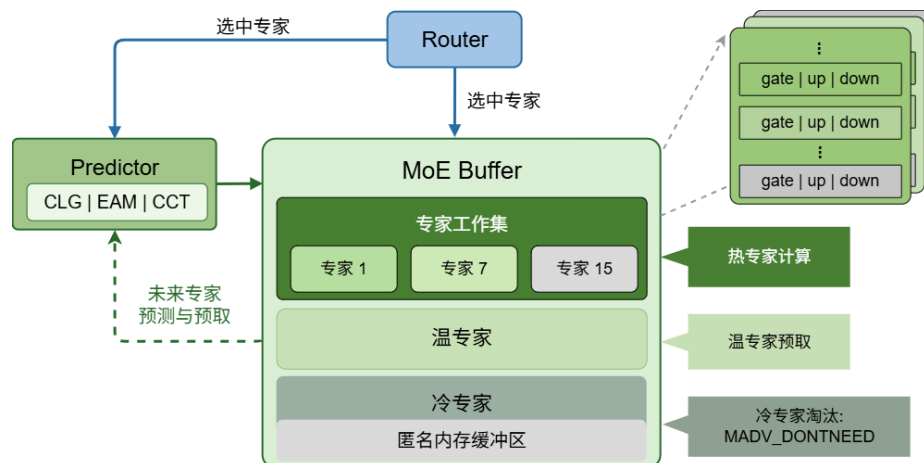


图 10: MoE Buffer 模块流程图

3.4.3 KV Cache 管理模块

KV Cache 是 LLM 自回归生成过程中的动态运行时状态，其占用量随上下文长度和并发 Session 数量持续增长，无法像模型权重那样静态分配。本系统将逻辑 KV 与物理驻留解耦，即一个 Session 的历史 KV 状态可逻辑存在，但不一定始终占用物理内存，从而在内存受限环境下实现弹性管理。

系统以固定大小的 Block 为基本管理单位，每个 Block 记录所属 Session、物理页状态、active/protected/shared 标记以及 generation 版本。通过严格的物理生命周期执行器，区分 dead/unowned 和 idle-live 两类可释放对象：前者直接执行 page-aligned 的 `madvise(MADV_DONTNEED)` 完成物理释放，后者则通过 OFFLOAD 将完整 cell-major 数据写入固定槽位的外部存储，成功后再释放物理页并标记为 SWAPPED。所有恢复操作均经过确定性校验，任何失败均在 graph 计算前阻断，确保 Attention 正确性不受影响。

在预算控制层面，Physical Budget View 提供权威的 resident/reclaimable 物理视图，并接入 V2 Elastic Resident Controller。该控制器以 soft target 为稳态目标，形成“采样物理视图 → 计算 resident excess → 优先 RELEASE dead/unowned → 按需 OFFLOAD idle-live → 重采样”的闭环，且 correctness-required PREFETCH 优先级高于 soft target，体现了驻留压缩与恢复代价间的权衡。

对于多个 idle live Session 之间的 victim 选择，V3 Cost-aware Hot/Cold 策略引入

基于历史反馈的代价评分：

$$\text{Score}_{V3}(s) = \frac{T_{\text{offload}}(s) + P_{\text{reuse}}(s) T_{\text{restore}}(s) + T_{\text{churn}}(s)}{\widehat{B}_{\text{exclusive physical}}(s)},$$

分数越低，表示每释放一字节所需付出的未来代价越小。该评分仅对通过安全过滤的合格候选者进行排序；若同一决策中任一合格候选者缺乏授权，则整体回退至 idle-age/LRU 策略，以避免混合不可比较的分数。恢复后附加短期租约保护，防止刚恢复的会话被立即再次卸载。

多会话策略评估基于 Alibaba trace 到真实服务器回放（replay）的统一链路，涵盖 ACTIVE、IDLE、REVISIT、TTL、DEAD、COLD_RESTART 等完整生命周期，并以精确完成预言机（Exact completion oracle）驱动。在此框架下比较 Resident、RELEASE-only、V2、idle-age 与 V3 等策略，记录 KV 常驻内存、回收字节数、恢复延迟及尾部延迟等指标。整体设计亦预留 active-latency/I/O feedback、Exact Prefix Cache、可选 cold-KV 压缩和 query-aware selective loading 作为扩展，分别用于后台 I/O 服务质量控制、跨请求前缀复用，以及在质量预算下进一步降低后备存储容量与传输量，而 correctness-required Exact restore 与图门控（graph gate）始终保持默认启用。

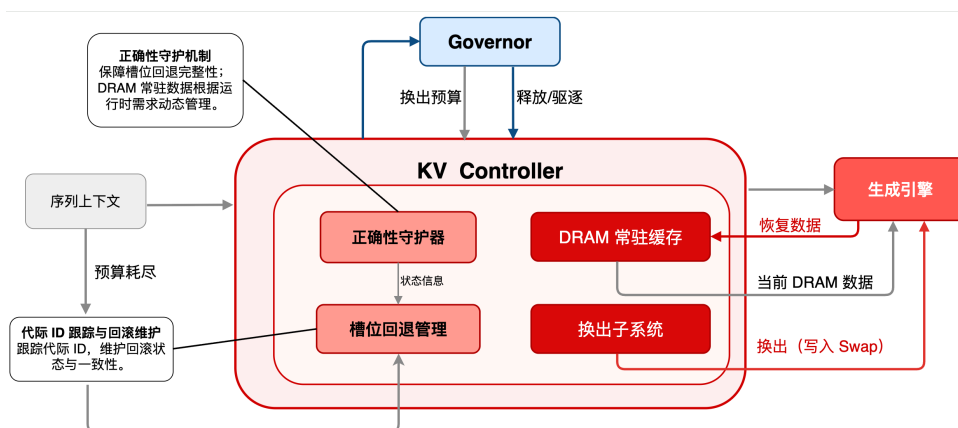


图 11: KV Cache 管理模块流程图

3.4.4 统一调度与执行机制

由于 Dense 权重、MoE 专家权重以及 KV Cache 均会竞争有限的内存容量和存储带宽，系统需要避免不同模块之间独立决策导致的资源冲突。本项目设计统一的运行时调度机制，由 Server Memory Governor 负责协调不同 Backend 的资源申请、释放和扩展行为。系统整体调度过程抽象为状态观测、资源决策、执行控制和效果反馈四个阶段构成的闭环。

状态观测阶段，系统周期性采集运行时状态，包括当前物理内存占用、cgroup memory pressure、Dense Layer 驻留情况、MoE Expert 工作集状态、KV Block 生命周期状态以及当前 I/O 负载。基于这些观测信息，Governor 在决策阶段判断系统当前处于正常运行状态、内存压力状态、I/O 受限状态或恢复扩展状态，并根据优化目标选择对应策

略。执行控制阶段，Governor 向不同 Backend 下发控制动作：Dense Flex 调整 Layer 驻留窗口，MoE Buffer 调整 Expert 工作集规模，KV Backend 执行 Block 回收或恢复，Prefetch Scheduler 调整加载任务。效果反馈阶段，系统根据实际执行结果更新资源状态，包括实际释放内存、缓存命中变化、I/O 延迟变化以及 Token 生成性能变化。通过上述闭环机制，系统从传统的请求驱动加载转变为主动观测、预测、调度、执行与优化的资源治理模式。

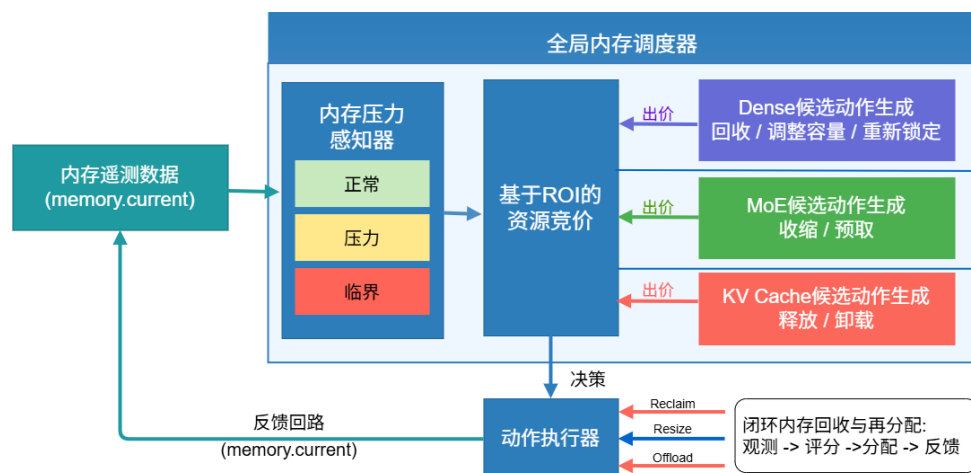


图 12: 统一调度模块流程图

3.5 系统设计总结

综上，本项目针对资源受限环境下 LLM 推理存在的内存容量不足和 I/O 延迟问题，设计了一套面向模型结构感知的运行时内存治理框架，实现了从被动内存管理到面向 LLM 的主动运行时内存治理的转变，为资源受限设备部署大规模语言模型提供了一种系统级优化方案。

4 模块设计与实现

本章从真实源码入口、核心数据结构、关键函数、运行时状态转换和安全边界五个维度说明系统如何落地。本项目不是在 llama.cpp 之外重新实现一套推理引擎，而是在其模型加载、GGML 计算图、CPU Backend、KV Cache 和 Server Scheduler 等既有路径上增加一层结构感知的显式驻留管理与全局内存治理机制。

从代码落点看，系统可以分为四条实现链：

- 加载期规划 (src/llama-model.cpp) 负责 cgroup 感知、模型类型识别、Dense/MoE 后端选择和初始预算规划；
- 权重执行路径 (src/llama-context.cpp、src/llama-flex.cpp、src/llama-moe-buffer.cpp) 实现 Dense Layer Streaming、MoE Expert Streaming 和 Tensor 指针重定向；

- KV 生命周期路径 (KV Memory Backend、tools/server/server-context.cpp) 完成 KV resident/reclaimable 观测、release/offload 及 slot-state fallback;
- 全局运行时治理 (tools/server/server-context.cpp) 涵盖 Pressure State、Candidate、Auction、Unified Prefetch、Async Action 和 Reallocation Credit。

4.1 加载期与推理期内存规划及后端选择

加载期 Memory Planner 负责在模型加载阶段根据 cgroup 约束、模型结构和硬件特性选择最优的显式驻留后端 (Dense Flex 或 MoE Buffer), 并为 Dense 和 MoE 分别制定细粒度的预算、Ring 大小、Ahead 层数和 Lock 策略。推理期则通过 CPU Hook 将后端接入计算图, 实现层权重的按需流式加载与释放。整个流程形成“规划 → 后端初始化 → 推理期回调”的闭环, 确保在有限内存下平衡性能与安全。

4.1.1 加载期自动后端选择

加载期统一入口位于 src/llama-model.cpp 的 llama_model_base::load_tensors。该函数首先读取 cgroup 状态并统计模型权重规模:

```

1  const auto planner_cgroup = llama_read_cgroup_memory_snapshot();
2
3  const bool memory_governor_auto_by_cgroup = planner_cgroup.valid &&
    planner_cgroup.max_bytes > 0;
4
5  const bool memory_governor_requested = llama_env_flag("
    LLAMA_MEMORY_GOVERNOR") || memory_governor_auto_by_cgroup;
6
7  size_t model_weight_bytes = 0;
8  size_t largest_weight_tensor_bytes = 0;
9
10 for (const auto & it : ml.weights_map) {
11     const size_t nb = ggml_nbytes(it.second.tensor);
12     model_weight_bytes += nb;
13     largest_weight_tensor_bytes =
14         std::max(largest_weight_tensor_bytes, nb);
15 }
16

```

随后, Planner 根据模型 Head 结构估算 KV 每 Token 的字节量:

```

1 size_t governor_kv_bytes_per_token = 0;
2
3 for (uint32_t il = 0; il < hparams.n_layer; ++il) {
4     governor_kv_bytes_per_token +=
5         ((size_t) hparams.n_head_kv(il) *
6         ((size_t) hparams.n_embd_head_k(il) +
7         (size_t) hparams.n_embd_head_v(il)) *
8         sizeof(uint16_t));
9 }
10

```

从系统设计角度，可表示为：

$$B_{KV/token} = \sum_{l=0}^{L-1} n_{head,kv}^{(l)} \left(d_k^{(l)} + d_v^{(l)} \right) b_{elem}$$

Planner 需要同时考虑 Weight Working Set、最大张量/运行时保护空间和 KV Reserve，而非简单地把全部可用 DRAM 都给权重缓存。

模型结构通过 `hparams.n_experts > 0` 判断，在启用 Auto Backend 时，Dense 模型走 Dense Flex，MoE 模型走 Lazy Window + MoE Buffer：

```

1 const bool model_has_moe = hparams.n_expert > 0;
2
3 const bool flex_requested = llama_env_flag("LLAMA_FLEX") || (
4     memory_governor_auto_backends &&!model_has_moe &&!llama_env_is_set("
5     LLAMA_FLEX"));
6
7 const bool use_flex = flex_requested &&!use_mlock &&!ml.check_tensors &&!
8     .use_mmap &&!params.vocab_only;
9
10 const bool use_lazy_window = !use_flex &&lazy_window_requested &&!
11     use_mlock &&!ml.check_tensors &&!ml.use_mmap &&!params.vocab_only;
12

```

其中 `!use_flex` 明确保证 Flex 与 Lazy Window 在同一次 loader pass 中互斥。同时，当 `use_lazy_window || use_flex` 时：

```

1 pimpl->cpu_buf_t_list = make_cpu_buf_t_list(devices, (use_lazy_window ||
2     use_flex)? false: params.use_extra_buf_t, params.no_host);

```

```

3 ml.init_mappings(!(use_lazy_window || use_flex),use_mlock ? &pimpl->
    mlock_mmmaps : nullptr);
4

```

该处理的意义是：当应用层已经建立显式 Weight Residency Backend 时，不再让传统 eager mapping / extra backend buffer 与之重复形成完整权重副本。这一点直接对应后续实验中 repack 导致 RSS 膨胀的问题。

4.1.2 Dense Planner v2

Dense Planner 不只是简单计算可用内存除以层大小，而是同时考虑非 Layer 固定权重、Runtime Guard、KV Reserve、最大 Layer Size、存储带宽、Compute Proxy、Ring、Ahead、Lock Budget 和 Risk-Aware Lock Search。Planner 遍历所有权重张量，将名称以 `blk.N` 开头的归入对应层，其余（如 embedding、output、final norm）归为 non-layer 固定内存地板：

```

1 std::vector<size_t> layer_bytes(hparams.n_layer, 0);
2 size_t non_layer = 0;
3 size_t layer_total = 0;
4
5 for (const auto & it : ml.weights_map) {
6     const size_t nb = ggml_nbytes(it.second.tensor);
7     int layer = -1;
8
9     if (std::sscanf(it.first.c_str(),
10         "blk.%d.", &layer) == 1 &&
11         layer >= 0 &&
12         layer < (int) hparams.n_layer) {
13         layer_bytes[layer] += nb;
14         layer_total += nb;
15     } else {
16         non_layer += nb;
17     }
18 }
19

```

随后根据存储带宽估算 I/O 时间与计算时间的比值 $\rho = T_{IO}/T_{compute}$ ，并据此决定初始 Ahead 和 Ring 大小：


```

1  const double io_ms =bw_mib_s > 0.0? ((double) layer_total /1048576.0) /
    bw_mib_s * 1000.0: 0.0;
2
3  const double rho =compute_ms > 0.0? io_ms / compute_ms: 0.0;
4
5  int planned_ahead = 1;
6  int planned_ring = 2;
7
8  if (rho < 0.5) {
9      planned_ahead = 3;
10     planned_ring = 5;
11 } else if (rho <= 2.0) {
12     planned_ahead = 2;
13     planned_ring = 4;
14 }
15

```

在 Lock 预算方面, Planner 为每层 Tensor 建立候选, 估计在不同 Lock Budget 下的 Locked Bytes 和 Stream Bytes, 并据此修正 Ring Slot 大小:

```

1  const double risk_limit = 0.88;
2
3  const size_t cgroup_cap =(size_t) ((double) planner_cgroup.max_bytes*
    risk_limit);
4
5  const size_t resident_estimate =planner_cgroup.current_bytes +soft_fixed
    +slot * (size_t) planned_ring +lock_budget;
6
7  dense_lock_risk_ratio =planner_cgroup.max_bytes > 0? (double)
    resident_estimate /(double) planner_cgroup.max_bytes: 0.0;
8

```

同时只有当新增 Lock 能带来足够 Streaming 收益时才接受:

```

1  const size_t saved =stream > cand_stream? stream - cand_stream: 0;
2
3  const size_t added =risk_lock_cap - lock_budget;
4
5  const bool useful =saved > 0 &&added > 0 &&saved * 4 >= added;

```

6

这种设计使 Planner 在低内存环境下兼顾安全性和效率。

4.1.3 MoE Budget Planner v2

MoE Planner 的核心问题与 Dense 不同：Dense 每 Token 需遍历全部 Layer，而 MoE 每层只访问部分 Expert，因此 Planner 更关注当前 `memory.max` 能否覆盖必要的 Expert Working Set。加载期通过张量名称含 `_exps` 且三维且第三维大于 1 来识别 Expert Tensor：

```
1 const bool is_exps =ggml_n_dims(t) == 3 &&t->ne[2] > 1 &&t.first.find("_exps") != std::string::npos;
```

2

成功注册至 MoE Buffer 后，该 Tensor 不再交给普通 Window 处理。

Planner 首先获取 `expert_bytes`、`warm_working_set`、`non_expert`、KV Reserve 和最大张量等指标：

```
1 const size_t expert_bytes =llama_moe_buffer_expert_bytes(pimpl->
    moe_buffer.get());
2
3 const size_t warm_working_set =llama_moe_buffer_warm_working_set_bytes(
    pimpl->moe_buffer.get(),moe_warm_coverage);
4
5 const size_t non_expert =model_total_bytes > expert_bytes?
    model_total_bytes - expert_bytes: 0;
```

6

在有限 `memory.max` 下，Planner v2 进一步构造 `soft guard`、`hard floor`、`legacy safe budget` 和 `working set floor`：

```
1 safe_budget = legacy_safe_budget;
2
3 if (safe_budget < moe_working_set_floor &&hard_safe_budget > safe_budget)
4     {
5         const size_t target =std::min(moe_working_set_floor,
6             hard_safe_budget);
7
8         safe_budget =std::max(safe_budget, target);
```

```
7 }
8
```

最终 budget 取 warm_working_set 与 safe_budget 的较小值:

```
1 const size_t budget =budget_unbounded? 0: std::min(warm_working_set,
    safe_budget);
2
3 llama_moe_buffer_set_budget_plan(pimpl->moe_buffer.get(),budget,
    budget_unbounded,safe_budget,moe_hard_floor);
4
```

这一机制正是后续实验中“1500M memory.max 下 auto_v2 成功运行”的代码基础。

4.1.4 推理期 CPU Hook: Dense 与 MoE 互斥接入

在 src/llama-context.cpp 中,系统通过获取 flex 和 moe 上下文,判断 flex_active 和 moe_active 是否有效,两者互斥:

```
1 auto * flex = model.get_flex_context();
2 auto * moe  = model.get_moe_buffer_context();
3
4 const bool flex_active =backend_cpu != nullptr &&llama_flex_enabled(flex)
    ;
5
6 const bool moe_active =backend_cpu != nullptr &&!flex_active &&
    llama_moe_buffer_enabled(moe);
7
8 if (flex_active) {
9     llama_flex_graph_begin(*flex);
10
11     ggml_cpu_set_weight_stream_callback(llama_flex_stream_callback,
        flex);
12
13     ggml_cpu_set_op_override_callback(nullptr, nullptr);
14
15 } else if (moe_active) {
16     ggml_cpu_set_weight_stream_callback(
```

```

17     llama_moe_buffer_stream_callback,moe);
18     ggml_cpu_set_op_override_callback(
19     llama_moe_buffer_mul_mat_id_callback,moe);
20 }

```

执行完成后同步并清除回调：

```

1 ggml_backend_sched_synchronize(sched.get());
2
3 ggml_cpu_set_weight_stream_callback(nullptr, nullptr);
4
5 ggml_cpu_set_op_override_callback(nullptr, nullptr);
6

```

该设计确认：Dense 只使用 Weight Stream Callback，MoE 额外允许 Op Override 进入 Sidecar/Fused Compute 路径。

4.2 Dense Flex: Layer 级显式权重驻留

Dense Flex 的核心目标是建立一个具有明确状态的 Layer Residency Backend，通过 Load-time Lock 和 Runtime Delta Pin 降低每 Token 反复读取的数据量。其数据流路径为：GGUF File → Background I/O → Layer Ring → Tensor Pointer Repoint → Compute → Release，形成一个完整的显式驻留管理闭环。

4.2.1 Dense Ring 的生命周期

Ring 作为固定容量的 Slot 池，设总 Slot 数为 N_{slot} ，当前被占用的 Slot 数为 N_{used} ，则 Ring 利用率定义为：

$$\rho = \frac{N_{\text{used}}}{N_{\text{slot}}}. \quad (1)$$

当 ρ 接近 1 时，新请求需驱逐旧层，其 Victim 选择策略依据最近最少使用（LRU）原则，但确保当前 Compute Layer 不被驱逐。

具体实现上，Slot 获取函数（`flex_acquire_slot`）首先寻找空闲 Slot，若无则选择已释放且状态为 `resident` 且 `last_use` 最旧的 Layer 作为 Victim，确保当前 Compute Layer 不会被驱逐：

```

1 for (size_t s = 0; s < ctx.slots.size(); ++s) {
2     if (ctx.slot_layer[s] < 0) {

```

```

3         ctx.slot_layer[s] = layer;
4         return (int) s;
5     }
6 }
7 // no free slot
8 for (int l = 0; l < ctx.n_layers; ++l) {
9     auto & L = ctx.layers[l];
10    if (L.slot >= 0 && L.released &&
11        L.state == layer_state::resident && L.last_use < oldest) {
12        victim = l;
13    }
14 }
15

```

后台 Worker (`flex_worker`) 循环从队列取任务：获取 Slot、标记 loading、读取未锁定张量、标记 resident，若无可安全 Slot 则 Requeue：

```

1 if (slot < 0) {
2     ctx->stats.queue_requeues++;
3     ctx->queue.push_back(layer);
4     std::this_thread::sleep_for(std::chrono::microseconds(200));
5     continue;
6 }
7

```

数据读取通过 `flex_read` 实现：普通路径直接使用 `pread`，`O_DIRECT` 模式下通过对齐的 Bounce Buffer 完成读取，并分别记录 `bytes_streamed` 和 `bytes_read_phys`，从而能够分析 `O_DIRECT` 对齐导致的真实物理读放大。其物理读放大因子可定义为：

$$\text{ReadAmplification} = \frac{\text{bytes_read_phys}}{\text{bytes_streamed}}. \quad (2)$$

4.2.2 Request-Wait-Get-Release 闭环

预取请求首先检查 Prefetch Budget。系统全局预取预算为 B_{prefetch} ，每个 Layer 的预取消耗等于其未锁定张量的字节数 s 。若 $s > B_{\text{prefetch}}$ ，则丢弃该请求并统计 `prefetch_budget_dropped`；否则消耗预算并提交。预算更新规则为：

$$B_{\text{prefetch}} \leftarrow B_{\text{prefetch}} - s. \quad (3)$$

Demand 路径将当前 Layer 推入队首并等待完成：

```

1 if (L.state == layer_state::not_resident) {
2     ctx.stats.demand_loads++;
3     ctx.queue.push_front(layer_id);
4     ctx.cv_work.notify_one();
5 }
6 ctx.cv_ready.wait(...)
7

```

llama_flex_get_tensor 的地址优先级为: Locked > Delta Locked > Ring Slot, 三条数据路径最终向 GGML 暴露统一的 tensor->data 指针。Release 操作仅将 Layer 标记为 released 并更新 last_use, 不立即释放物理页。

4.2.3 Balanced Locking 与 Pin Policy

llama_flex_finalize 中保证每层获得相近的 Lock Budget ($\text{lock_bytes}/n_{\text{layers}}$), 避免某些层被完全 Pin 而另一些层每 Token 仍需大量 Streaming。Pin Policy 只决定同一预算内哪些 Tensor 值得常驻:

- **非 Cost-aware 路径:** 使用 flex_sort_for_pin, 按张量大小降序选择。
- **Cost-aware 路径:** 使用 flex_choose_cost_aware_pins 计算 ROI 选择最优。对于候选张量 t , 其 ROI 定义为:

$$ROI(t) = \frac{\text{Value}(t)}{\text{Cost}(t)}, \quad (4)$$

其中 $\text{Cost}(t)$ 通常为张量字节数, $\text{Value}(t)$ 可基于访问频率或延迟敏感度计算。系统选择 ROI 最高的张量直至预算用尽。

最终统计 stream_per_token 作为 Load-time Pin Strategy 的直接机制指标。

Adaptive Ahead 在每次 Layer 边界变化时根据 EWMA 的 I/O、计算和等待时间调整有效 Ahead。设当前 Ahead 层数为 A , 调整量为 ΔA , 则更新规则为:

$$A \leftarrow \text{clamp}(A + \Delta A, A_{\min}, A_{\max}), \quad (5)$$

其中 ΔA 由实测 I/O 延迟与计算延迟之比决定, 且受 Ring Room (空闲 Slot 数) 和 Unified Prefetch Budget 限制。

Clean Reclaim (llama_flex_reclaim_released) 对已消费完成的 Layer Slot 执行 madvise(MADV_DONTNEED), 且跳过当前计算层。回收的字节数 R_{clean} 累加到 Governor 的 Raw Relieved 统计中。

Runtime Ring Resize (`llama_flex_resize_ring`) 支持 Grow (分配新 Slot) 和 Shrink (仅删除 Free Slot), 是 Best-effort、Non-destructive 操作。调整后的 Slot 数 N'_{slot} 满足:

$$N'_{\text{slot}} = N_{\text{slot}} + \Delta N, \quad (6)$$

其中 ΔN 可为正或负, 但保证不会释放正在使用的 Slot。

Delta Pin (`llama_flex_delta_pin_async`) 将回收所得内存重新投资于高 ROI Tensor。候选张量 t 的筛选条件为:

- 未锁定 (包括 `delta_locked`)
- 尺寸 > 0
- 字节数 $\leq$ 可用预算
- ROI 不低于阈值 θ (例如 0.8 倍平均 ROI)

候选按 ROI 降序、再按大小、层、名称排序。ROI 计算方式与 Cost-aware Pin 一致。若已有 Delta Pin Job 则拒绝新任务, 避免无控制叠加, 成功提交后由独立 `delta_pin_worker` 处理 I/O。该机制对应 Governor 的“内存再投资”路径。

综上, Dense Flex 通过 Ring 管理、预算控制、自适应预取和 ROI 驱动的 Pin/Delta Pin, 实现了显式、量化的权重驻留管理。

4.3 MoE Buffer: Expert 级显式驻留

MoE Buffer 与 Dense Flex 的核心区别在于管理粒度: Dense 以 Layer 或 Tensor 为基本单位, 而 MoE 以 $(\text{layer}, \text{expert})$ 即 Expert Slice 为基本管理粒度。MoE Buffer 为完整 `*_exps` Tensor 建立逻辑上等尺寸的匿名虚拟 Buffer, 但只有实际加载过的 Expert Slice 形成物理页驻留, 即:

$$\text{Virtual Expert Tensor} \neq \text{Physical Expert Working Set}.$$

这种设计使得系统在逻辑上保留完整的专家参数布局, 而物理内存中仅驻留实际被访问的专家切片。

4.3.1 Warm Working Set Estimator

Warm Working Set Estimator (`llama_moe_buffer_warm_working_set_bytes`) 为每个候选 Group 计算:

$$\text{Prior} = \text{seq_access} + \text{access} + \text{hot_score},$$

$$\text{Reuse Distance} = \text{inter_token_gap_ema},$$

$$\text{Gain} \approx \frac{\text{prob} \times \text{group_bytes}}{\text{reuse_distance}}.$$

无历史时退化为 Uniform Prior。随后每层按 Gain 排序，累计到 warm_coverage 阈值，从而得到根据 Group Probability、Size 与 Reuse 共同估算的 Resident Set。

4.3.2 MoE Predictor: CLG、EAM 与 CCT

CLG (Cross-Layer Gate): 加载期 Window 与 MoE Buffer 建立连接:

```
1 llama_window_set_moe_buffer(*pimpl->window, pimpl->moe_buffer.get());
2
```

使得 CLG 预测结果直接转为 MoE Buffer 的异步预取请求。

EAM (Expert Access Model): moe_eam_predict_after_route() 在 Router 结果后更新转移矩阵，预测未来层 Expert:

$$P(e_{l+d} | e_l).$$

转移概率矩阵 $\mathbf{T}$ 按层维护，其元素 $T_{i,j}^{(l,d)}$ 表示在层 l 访问专家 i 后，在 d 层后访问专家 j 的频率。每次实际路由后，相应计数增加:

$$C_{i,j}^{(l,d)} \leftarrow C_{i,j}^{(l,d)} + 1,$$

然后归一化得到概率:

$$T_{i,j}^{(l,d)} = \frac{C_{i,j}^{(l,d)}}{\sum_k C_{i,k}^{(l,d)}}.$$

预测时，计算每个候选专家的 Score，若 Score 低于阈值（默认 75.0）则跳过:

```
1 if (score < moe_env_f64("LLAMA_LAZY_MOE_EAM_PREDICT_MIN_SCORE", 75.0))
   continue;
2
```

通过阈值的专家进入 Ranked Prefetch Queue。

CCT (Confidence-Controlled Transition): 分支预测风格的饱和置信度，用于 Eviction Protection 和 Retention，而非直接发起预取。

4.3.3 MoE Group-level Eviction

Victim Score 综合考虑多个因子，可抽象为加权线性组合:

$$\text{VictimScore}(g) = \sum_k w_k \cdot f_k(g),$$

其中 f_k 包括: future distance、recent use、hot score、high-sequence protection、active window、reuse、bad reload、ghost reload、CCT、EAM、bit width、speculative-unused state、layer locality 和 pressure。具体加权系数由运行时参数决定。其中 Pressure-Aware Spec Guard 体现为:

```

1 const double pressure_scale = ctx.params.pressure_scale_spec_guard ?
    moe_pressure_scale_locked(ctx) : 0.0;
2
3 const double spec_guard_penalty = speculative_unused &&!spec_evictable
    ? 4.0e8 * (1.0 - pressure_scale) : 0.0;
4

```

当压力升高时, 对 speculative-unused 对象的保留惩罚降低, 使其更易被驱逐。

4.3.4 MoE Pressure-Aware Protection

Pressure-Aware 保护允许压力动态作用于 pin fraction、layer cap、active window、cooldown 和 speculative guard。其核心为压力比例计算:

$$\text{ratio} = \frac{\text{resident_bytes}}{\text{budget_bytes}}, \quad \text{scale} = \begin{cases} 0, & \text{ratio} \leq \text{soft}, \\ \frac{\text{ratio} - \text{soft}}{\text{hard} - \text{soft}}, & \text{soft} < \text{ratio} < \text{hard}, \\ 1, & \text{ratio} \geq \text{hard}. \end{cases}$$

有效参数按比例缩放, 例如:

$$\text{effective_pinned_fraction} = \text{pinned_fraction} \times (1 - \text{scale}),$$

但保留最小下限 (floor)。该机制防止保护策略在高压下锁死 Resident Set。

4.3.5 MoE Clean Reclaim 与 Dynamic Budget

Clean Reclaim: llama_moe_buffer_reclaim_clean 回收 Cold Resident Expert-Group, 通过 MADV_DONTNEED 释放物理页。回收量 R_{moe} 累加到全局 Raw Relieved 统计, 并计入 Reallocation Credit 的 Pending 环节。

Runtime Budget: llama_moe_buffer_set_budget 允许 Governor 控制下动态 shrink/grow。设当前预算为 B_{moe} , 新预算为 B'_{moe} , 则:

$$B'_{\text{moe}} = \text{clamp}(B_{\text{moe}} + \Delta B, B_{\min}, B_{\max}),$$

其中 ΔB 由 Reallocation Credit 分配的 $\Delta \text{Budget}_{\text{moe}}$ 决定 (参见前文积分系统)。Shrink 操作仅释放空闲 Slot, 不影响已驻留专家; Grow 操作尝试分配新匿名 Buffer, 用于未来加载。

4.4 KV Cache 运行时内存管理

KV Cache 与模型权重的本质区别在于：权重具有稳定的文件后备，而 KV 是随请求运行时生成、并与 Session 历史语义绑定的状态数据。因此，KV 不能仅依赖操作系统被动回收文件页，而需要同时管理逻辑历史、物理驻留、后备副本、恢复成本与会话生命周期。本系统将 KV 子系统划分为三层：**Exact 物理生命周期执行层**、**Physical Resident Budget 控制层**、**Cost-aware Hot/Cold 驻留选择层**。其中 KV core 负责 block/cell ownership、状态转换、backing I/O、transaction/generation 与真实物理动作，Server/Governor 负责 Session 生命周期、resident budget 与 claimant policy。

4.4.1 Block/Cell 生命周期与 Unified Action

系统以 block 为物理执行粒度、以 cell 为 KV 数据布局粒度，维护 UNUSED/RESIDENT/RELEASED/SWAPPED/PENDING_WRITE/INVALID 等状态。每个 Sequence 通过 seq/epoch 绑定逻辑历史，每次物理动作同时携带 object/generation，用于识别 cache object 重建、mapping 更新或旧事务完成事件。RELEASE 用于销毁 dead/unowned KV 的物理页和逻辑占用；OFFLOAD 用于将仍可能复用的 idle live KV 写入固定槽位 backing 后回收原物理页；PREFETCH 则将 SWAPPED block 恢复到原 KV tensor。

为了避免 Server 直接修改 KV 内部 block 状态，系统将所有 destructive/restore 动作收敛到统一 `execute_action()` 接口。Core 首先建立 capability，再对 write transaction、sequence、protection、ownership 与 block state 做二次校验。OFFLOAD 路径的关键安全检查如下：

```

1  if (!result.capability.can_offload) {
2      result.outcome = llama_kv_action_outcome::unsupported;
3      return result;
4  }
5  if (request.seq_id < 0 ||
6      (size_t) request.seq_id >= seq_to_stream.size()) {
7      result.outcome = llama_kv_action_outcome::rejected;
8      result.reason = llama_kv_action_reason::invalid_sequence;
9      return result;
10 }
11 if ((size_t) request.seq_id < paged_prefetch_protected_seq.size() &&
12     paged_prefetch_protected_seq.test(request.seq_id)) {
13     result.outcome = llama_kv_action_outcome::rejected;
14     result.reason = llama_kv_action_reason::protected_sequence;
15     return result;
16 }

```

17

随后 Core 根据 logical-to-physical mapping 收集目标 block; 一旦发现 shared ownership、非法映射或非 RESIDENT/SWAPPED 状态即拒绝动作。实际换出只有在 backing 写入成功后才发布 SWAPPED, 并通过页对齐的 `madvise(MADV_DONTNEED)` 释放可安全回收的物理页。这样保证了“先获得可恢复副本, 再回收原页”的事务顺序, I/O 失败时不会把未完整落盘的 KV 错误标记为可恢复状态。

4.4.2 Physical Budget View 与 RELEASE-first 控制闭环

KV resident budget 直接约束 KV tensor 的真实物理驻留。Core 通过 `sample_kv_physical_budget_view()` 形成同一 object/generation 下的 coherent snapshot, 汇总 resident bytes、dead resident reclaimable bytes、authoritative swapped payload、owned/shared block 数以及 transient staging bound。Resident 与 reclaimable 分别复用现有物理采样和 release-budget authority, 保证不同模块采用一致的物理内存统计口径:

```

1  const auto resident_sample = sample_kv_resident();
2  if (resident_sample.available) {
3      result.resident_available = true;
4      result.resident_bytes = resident_sample.resident_bytes;
5  }
6
7  const auto release_budget = sample_kv_release_budget();
8  if (release_budget.valid) {
9      result.reclaimable_available = true;
10     result.dead_resident_reclaimable_bytes =
11         release_budget.reclaimable_resident_bytes;
12 }
13
14 result.valid = true;
15
```

Soft Resident Target 下的预算缺口定义为:

$$\text{Excess}_{KV} = \max(0, B_{\text{resident}} - B_{\text{target}}). \quad (7)$$

当 Excess 大于 0 时, Governor 采用 **RELEASE-first**: 优先回收 dead/unowned KV, 因为这类数据无需 backing I/O, 也不存在后续恢复成本; 只有当 RELEASE 返回 no-candidate 且 resident 仍高于 target 时, 才在后续 decision 中 arm OFFLOAD, 从 idle

live claimant 中选择一个受控换出。每次状态改变后都重新读取 Physical Budget View，而不是将理论 action bytes 直接当成已经到账的物理内存收益。

这种分层控制把释放量与被释放者分开：Budget View 决定当前是否存在 resident debt，RELEASE/OFFLOAD executor 决定能安全回收哪些 block，而 claimant policy 仅在多个合法 idle Session 之间决定 victim。

4.4.3 Cost-aware Hot/Cold Claimant Ranking

当多个 idle live claimant 同时存在时，单纯按 LRU 或逻辑 KV 大小驱逐容易出现两类问题：一是驱逐对象虽然逻辑容量大，但实际独占驻留页较少，物理收益有限；二是刚刚恢复或很快会再次访问的 Session 被反复 OFFLOAD/PREFETCH，产生额外写盘、restore 与 page population 开销。因此系统为每个 claimant 构建 decision-scoped feature view，结合当前独占物理可释放量、历史 OFFLOAD 写成本、restore gate、reuse 信号与 round-trip churn 计算单位物理字节的未来迁移代价。

评分逻辑可表示为：

$$\text{Score}(s) = \frac{T_{\text{offload}}(s) + P_{\text{reuse}}(s) T_{\text{restore}}(s) + T_{\text{churn}}(s)}{\hat{B}_{\text{exclusive}}(s)}, \quad (8)$$

其中分母 $\hat{B}_{\text{exclusive}}$ 来自 claimant 级 mincore/ownership 聚合，不使用 logical KV size 冒充 physical relief。对应源码中的核心计算等价于：

```

1  const uint64_t expected_cost_us =
2  history.last_offload_time_us +
3  reuse_probability_ppm * history.last_restore_gate_us / 1000000ULL +
4  params.churn_penalty_us * history.round_trip_count;
5
6  score.total = expected_cost_us * 1000000ULL /
7  physical.estimated_exclusive_resident_bytes;
8

```

Score 越低，表示“释放 1 Byte 真实物理内存所付出的未来代价”越低，因此优先被驱逐。安全 eligibility 始终先于排序：active、protected、shared、epoch/generation 不一致以及处于 resident lease 的 claimant 不参与 destructive action。若任一 eligible claimant 缺少 authoritative physical estimate、write cost、restore cost 或 reuse evidence，则整个 decision 统一采用 idle-age/LRU authority，避免将不同量纲的 score 混合排序。

为了抑制 OFFLOAD–restore–OFFLOAD 的往返抖动，系统在成功 restore 后设置 resident_lease_until_sample，lease 内禁止普通 OFFLOAD；每次真实 OFFLOAD 后再 restore 会增加 round_trip_count，并通过 churn penalty 提高下一次驱逐成本。普通 turn rollover 迁移 claimant history，只有 prompt clear、object/generation 变化等真实 lineage loss 才清空历史，从而使成本反馈能够跨多轮会话持续生效。

4.4.4 Exact PREFETCH 与 Graph Gate

Session 重访时，恢复动作不依赖 Governor 的 soft victim policy，而是进行独立的 correctness-required PREFETCH。Server 在 `n_past` 确定之后、batch/graph 构建之前保护目标 sequence，并提交 all-required restore；只有 decision identity、I/O、fail-stop、context 与 shortfall 全部满足要求时，才允许后续 `llama_decode()`：

```

1 ops.set_protected(seq_id, true);
2 llama_kv_action_request request;
3 request.action = llama_kv_action::prefetch;
4 request.decision_id = decision_id;
5 request.seq_id = seq_id;
6 request.max_blocks = 0;
7 request.correctness_required = true;
8 request.all_required = true;
9 result.action = ops.execute(request);
10
11 result.graph_allowed =
12 result.action.decision_id == decision_id &&
13 (result.action.outcome == llama_kv_action_outcome::completed ||
14 result.action.outcome == llama_kv_action_outcome::no_op) &&
15 !result.action.io_failure &&
16 !result.action.fail_stop &&
17 !result.action.capability.context_invalid &&
18 result.action.shortfall_bytes == 0;
19

```

这条路径保证 policy prediction 只影响性能，不影响正确性：即使前面的 Hot/Cold 判断没有提前保留某个 Session，真实请求到达后仍由 Exact gate 恢复所有 required SWAPPED blocks；partial failure、I/O error 或 shortfall 均会阻断 graph，而不是带着不完整 KV 继续推理。

4.4.5 Transfer Group、Read/Restore Pipeline 与 Prefault

为了降低 Exact restore 的 I/O 与 page population 开销，系统不再按单 block 独立完成完整 read-scatter-commit，而是先将连续 SWAPPED block 合并为 bounded transfer group。Group 内记录 seq/object/generation/mapping generation、begin/end block、cell range 与 total bytes；只有连续 block 且累计字节不超过 group cap 时才合并：

```

1 const bool start_group = tasks.empty() ||

```

```

2 block != tasks.back().end_block ||
3 tasks.back().total_bytes > byte_cap ||
4 block_bytes > byte_cap - tasks.back().total_bytes;
5
6 if (start_group) {
7     paged_restore_group_task task;
8     task.seq_id = seq_id;
9     task.object_id = paged_resident_object_id;
10    task.generation = paged_resident_generation;
11    task.mapping_generation = paged_mapping_generation;
12    task.begin_block = block;
13    tasks.push_back(std::move(task));
14 }
15

```

在此基础上，restore 采用有界的两级流水：当前 group 完成 read 后，后台 read worker 提前读取下一 group 到 task-owned staging；scheduler owner 同时对当前 group 执行 identity revalidate、destination prefault、tensor scatter 与 RESIDENT commit。核心调度关系为：

```

1 paged_restore_group_start_async(current);
2 paged_restore_group_wait_async(*current);
3
4 for (size_t i = 0; i < restore_tasks.size(); ++i) {
5     if (i + 1 < restore_tasks.size()) {
6         paged_restore_group_prepare(*next);
7         paged_restore_group_start_async(next); // READ(N+1)
8     }
9
10    paged_restore_group_post_read(*current); // RESTORE(N)
11    paged_restore_group_complete(*current);
12    std::swap(current, next);
13 }
14

```

Read worker 只负责 backing read 与 task-owned staging，不直接修改 live KV tensor；scatter、状态发布、transaction 与 graph gate 始终由 scheduler owner 完成。这样既能形成 READ(N+1)||RESTORE(N) 的 I/O/恢复重叠，又避免异步 worker 与 active graph 对

同一 KV tensor 并发写入。目标页 prefault 在 scatter 前主动建立目的页，进一步减少 restore 期间的同步缺页抖动。

综上，KV 运行时形成统一闭环：Physical Budget View 判断需要释放多少内存，RELEASE-first 区分不可恢复与可恢复回收，Cost-aware Hot/Cold 决定优先驱逐哪个 Session，Unified Action 在 core 内执行 block 级安全状态转换，Exact PREFETCH 与 Transfer Group Pipeline 负责恢复，并由真实 physical relief、restore cost、lease/churn 反馈下一轮决策。这一设计使容量控制、victim selection、I/O 恢复与 correctness gate 共享同一套 block/cell 生命周期和物理身份。

4.4.6 Cost-aware Hot/Cold 策略

在多个 idle live Session 同时竞争有限 KV resident budget 时，FlexKV-OS 使用 Cost-aware Hot/Cold Ranking 选择 OFFLOAD victim。该策略不依赖静态的 HOT/COLD 标签，而是在每次决策时根据 claimant 的实时物理驻留量和历史迁移代价计算驱逐成本。

对于 claimant s ，核心评分可写为：

$$Score(s) = \frac{T_{\text{offload}}(s) + P_{\text{reuse}}(s) \cdot T_{\text{restore}}(s) + T_{\text{churn}}(s)}{\hat{B}_{\text{physical}}(s)}$$

其中：

- $\hat{B}_{\text{physical}}$ 为基于 mincore 和 ownership 统计得到的 exclusive resident estimate；
- T_{offload} 来自该 Session 上一次真实 OFFLOAD action 的 wall time；
- T_{restore} 来自该 Session Exact Resume 的真实 gate time；
- P_{reuse} 描述 KV 再次被访问的概率；
- T_{churn} 根据 OFFLOAD $\leftrightarrow$ restore 往返历史增加抖动惩罚。

Score 越低，表示“每释放一个真实物理字节所付出的预期代价越低”，因而具有更高的 OFFLOAD 优先级。ACTIVE、protected、shared、stale-generation 等 claimant 在评分前由 safety gate 直接排除；restore 后的 resident lease 则用于避免刚恢复的 KV 被立即再次驱逐。

对于物理驻留或历史成本信息不足的决策，系统统一切换到 idle-age authority，避免不同量纲的 score 在同一次排序中混用。OFFLOAD 完成后，实际 physical relief、write cost 和后续 restore gate 会重新写入 claimant history，使下一次决策能够基于真实执行反馈更新 Hot/Cold 排序。

4.5 Server Memory Governor

Server Memory Governor 的输入信号包括：cgroup 内存状态（memory.current / memory.max / memory.high）、RSS 常驻内存、KV Cache 压力、Dense 模型 Flex 统计、MoE 缓冲区统计、KV 预算、分配动作历史、冷却状态以及重分配信用额度，输出 Dense/MoE 的回收、预算及预取策略，KV 释放/卸载，重分配与观测标记动作。

Candidate 结构体统一携带 kind、action、id、score、roi、bytes、reason 等字段，使 Dense Layer、MoE ExpertGroup、KV Global 和 KV Sequence 进入同一决策接口：

```

1 struct memory_governor_candidate {
2     const char * kind = "none";
3     const char * action = "none";
4     int32_t id = -1;
5     int64_t score = std::numeric_limits<int64_t>::min();
6     double roi = -std::numeric_limits<double>::infinity();
7     uint64_t bytes = 0;
8     const char * reason = "none";
9 };
10

```

4.5.1 Pressure Gate

Pressure Gate 在 Auction Select 中首先检查候选是否允许在当前压力状态下执行。系统根据当前 Cgroup 内存使用量 M_{cur} 、软限制 M_{high} 和硬限制 M_{max} 定义压力指标 P ：

$$P = \frac{M_{\text{cur}} - M_{\text{high}}}{M_{\text{max}} - M_{\text{high}}}, \quad \text{其中当 } M_{\text{cur}} \leq M_{\text{high}} \text{ 时令 } P = 0. \quad (9)$$

压力状态映射为：

- **NORMAL:** $P < 0$ （内存充裕）
- **PRESSURE:** $0 \leq P < 0.6$
- **CRITICAL:** $P \geq 0.6$

不同状态允许执行的动作集合不同：

$$\text{PermittedActions}(P) = \begin{cases} \{\text{prefetch, reserve}\}, & P < 0 \quad (\text{NORMAL}) \\ \{\text{reclaim, resize}\}, & 0 \leq P < 0.6 \quad (\text{PRESSURE}) \\ \{\text{release, offload, re-pin}\}, & P \geq 0.6 \quad (\text{CRITICAL}) \end{cases} \quad (10)$$

候选 c 通过门控的条件为 $\text{Action}(c) \in \text{PermittedActions}(P)$ 。

4.5.2 Auction Select 与候选排序

通过门控的候选将按综合效用降序排列，并累计字节数直到达到 `warm_coverage` 阈值。

定义候选的回报率（ROI）为：

$$ROI(c) = \frac{\text{score}(c)}{\text{bytes}(c) + \epsilon}, \quad (11)$$

其中 ϵ 为极小正数防止除零。综合排序分采用调和形式：

$$U(c) = \alpha \cdot \text{score}(c) + \beta \cdot ROI(c) \cdot \text{bytes}(c), \quad (12)$$

默认取 $\alpha = 0, \beta = 1$ 即纯 ROI 排序。

4.5.3 Async Action Executor

异步执行器覆盖 `dense_clean_reclaim`、`moe_clean_reclaim`、`kvs_global_release`、`kvs_sequence_offload` 和 `kvs_sequence_slot_state_offload`，通过原子 Inflight Flag 防止同类动作重复并发：

```

1 if (!inflight.compare_exchange_strong(...)) {
2     memory_governor_async_rejected++;
3     return false;
4 }
5

```

4.5.4 Unified Prefetch Budget

系统维护全局 Tick Budget B_{total} ，按优先次序分配。KV Resume 具有最高优先权，通过 `memory_governor_prefetch_budget_reserve` 预留：

$$B_{\text{kv_reserve}} = \min(\text{Demand}_{\text{kv}}, B_{\text{total}} \times \gamma), \quad (13)$$

其中 γ 为 KV 预留比例上限（例如 0.5）。剩余预算为 $B_{\text{remain}} = B_{\text{total}} - B_{\text{kv_reserve}}$ 。

剩余预算按动态比例 λ 分配给 Flex（Dense）和 MoE Buffer：

$$B_{\text{flex}} = \lambda \cdot B_{\text{remain}}, \quad (14)$$

$$B_{\text{moe}} = (1 - \lambda) \cdot B_{\text{remain}}, \quad (15)$$

其中 λ 根据当前失效率动态调整：

$$\lambda = \frac{\text{FlexMissRate}}{\text{FlexMissRate} + \text{MoEMissRate}}. \quad (16)$$

4.5.5 Clean Reclaim

统一调用包括 Dense 的 `llama_flex_reclaim_released`、MoE 的 `llama_moe_buffer_reclaim_clean` 和 KV 的 `execute_action(release/offload)`。KV 不纳入 `clean_reclaim` API，因其需要保持运行状态语义。

4.5.6 Reallocation Credit

系统将本轮策略调整后获得的内存释放量作为分配来源。该值首先进入挂起状态，随后通过当前内存使用确认环节，结合 `cgroup` 实时内存占用 `memory.current` 进行核实，确保释放量真实反映在空闲内存中。通过确认的部分转化为可分配内存。

为防止内存累积过快或长期空闲，系统引入指数衰减，随时间降低旧空闲内存价值，并辅以保护机制防止过度再分配引发 OOM。最终，经过以上处理的空闲内存被用于再分配。具体量化如下：

设第 t 步的原始释放量为 $R_{\text{raw}}(t)$ ，但物理内存变化需通过 `memory.current` 确认：

$$\text{Pending}(t) = \max(0, -\Delta M_{\text{cur}}(t)), \quad (17)$$

其中 $\Delta M_{\text{cur}}(t) = M_{\text{cur}}(t) - M_{\text{cur}}(t-1)$ 为内存变化（下降为负）。

实际空闲内存采用指数衰减：

$$\text{Earned}(t) = \text{Earned}(t-1) \cdot (1 - \delta) + \text{Pending}(t), \quad (18)$$

$\delta \in [0.05, 0.2]$ 为衰减率。

为防止过度分配，设置上限和守护条件：

$$C(t) = \min(\max(\text{Earned}(t), 0), \text{Cap}), \quad (19)$$

并仅当压力 $P < 0.3$ 时允许分配：

$$\text{InvestAllowed} = \mathbf{1}_{P < 0.3}. \quad (20)$$

可分配的内存 $C(t)$ 按固定比例 $\mu_{\text{moe}}, \mu_{\text{dense}}$ 分流：

$$\Delta \text{Budget}_{\text{moe}} = \mu_{\text{moe}} \cdot C(t), \quad (21)$$

$$\Delta \text{Budget}_{\text{dense}} = \mu_{\text{dense}} \cdot C(t), \quad (22)$$

$$\Delta \text{Budget}_{\text{prefetch}} = (1 - \mu_{\text{moe}} - \mu_{\text{dense}}) \cdot C(t). \quad (23)$$

4.5.7 Observation Marker

输出分为 Planner、Pressure、Dense、MoE、KV、Global 六类关键指标，其中：

- Pressure：压力 P 及其滑动平均 $\text{EMA}(P) = 0.9 \cdot \text{EMA}_{\text{prev}} + 0.1 \cdot P$

- Dense / MoE: 当前 Resident Set 覆盖率 $Coverage = \frac{\sum bytes_{selected}}{\sum bytes_{total}}$
- KV: Offload 比例 $OffloadRatio = \frac{Bytes_{offloaded}}{Bytes_{total_kv}}$
- Global: 内存松弛度 $Slack = \frac{M_{high} - M_{cur}}{M_{high}}$

这些观测为系统自适应调优提供运行期间数据。

5 系统测试

5.1 测试环境

本项目所有实验均在统一物理环境下完成，以保证不同优化模块之间的可比性。测试过程中分别启用或关闭权重管理、MoE Buffer、KV Cache 管理以及调度优化模块，通过环境变量控制运行模式，无需切换推理框架。

测试环境参数如表 5 所示。

表 5: 测试环境参数

环境指标	环境参数
Linux 发行版	Ubuntu 22.04
Linux 内核	5.15.0+
机器模式	物理机
内存大小	23GB
处理器	x86-64, 8 核
存储设备	SSD, 顺序读速约 279MB/s
Dense 测试模型	Llama-3-8B-Instruct-Q4_K_M.gguf
Dense 模型大小	约 4.58GB
MoE 测试模型	Qwen1.5-MoE-A2.7B-20-experts-SFT-trained.Q4_K_M.gguf
MoE 模型结构	24 个 MoE layer, 每层 20 个 expert, top-4 激活
内存限制方式	cgroup v2 (memory.max)
测试工具	llama-bench、perplexity、trace replay 工具

本项目测试主要围绕三个目标展开：

1. 验证系统是否能够降低大模型推理过程中的物理内存占用；
2. 验证权重流式加载、缓存驻留以及运行时调度机制是否能够降低 I/O 等待；
3. 验证优化路径是否保持与原始 mmap 推理路径一致的语义。

5.2 Dense 模型权重管理模块测试结果

权重管理模块主要针对 Dense 与 MoE 模型推理过程中权重物理驻留不可控的问题，通过 Repack 消除、Window/Flex Buffer、Risk-Aware Lock、Pin Policy 以及 Adaptive Ahead 等机制实现运行时权重管理。

实验主要从以下三个方面展开：

1. 权重副本消除带来的基础 RSS 降低；
2. Dense 模型在不同驻留策略下的权重流式性能；
3. 低内存环境下权重预算选择与 I/O 调度能力。

5.2.1 Repack 优化与基础内存降低

首先测试原生路径中的 repack 操作对物理内存占用的影响。传统 mmap 推理路径为了适配计算布局，通常需要额外创建匿名权重副本。然而，在本系统中，Dense 路径通过 Window + Flex Buffer 实现局部权重驻留，MoE 路径通过 Expert-level Buffer 直接管理专家权重，此时原始 repack 中间副本成为冗余结构。

实验结果如表 6 所示。

表 6: Repack 优化对基础内存占用的影响

配置	RSS (GB)	吞吐 (tok/s)	说明
Dense native (含 repack)	7.77	8.15	mmap + repack 匿名副本
Dense (关闭 repack)	4.56	8.60	消除额外权重复制
MoE native (含 repack)	5.73	19.26	expert 全量驻留
MoE (关闭 repack)	3.60	19.58	Buffer 直接管理 expert 权重

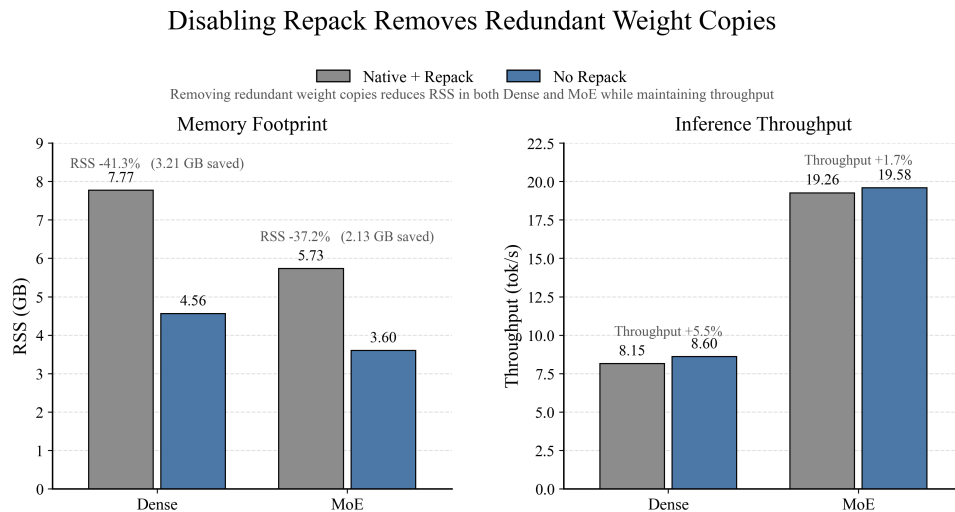


图 13

实验结果表明，关闭 repack 后，Dense 模型 RSS 从 7.77GB 降低至 4.56GB，减少约 3.21GB；MoE 模型 RSS 从 5.73GB 降低至 3.60GB，减少约 2.13GB。与此同时，两种模型的吞吐均未下降，Dense 吞吐由 8.15 tok/s 提升至 8.60 tok/s，MoE 吞吐由 19.26 tok/s 提升至 19.58 tok/s。

该结果说明，传统 repack 路径引入的匿名权重副本是端侧大模型推理中的重要内存开销来源。通过运行时权重布局管理，可以在不改变计算流程的情况下消除该部分额外占用，为后续 Residency 与 Buffer 调度提供更大的可用内存空间。

5.2.2 Dense 模型权重驻留机制测试

Dense 模型具有逐层顺序执行特点，但每个生成 token 均需要访问完整模型权重。因此，在内存受限环境下，如果无法保持有效权重驻留集合，系统会退化为频繁磁盘加载模式。本系统针对 Dense 模型设计 Window + Flex Buffer 权重驻留机制，仅保持当前计算窗口附近层权重常驻，其余层根据访问顺序进行异步加载。

不同权重驻留策略下的主要实验结果如表 7 所示。

表 7: Dense Residency 机制效果

配置	Stream/token	Flex Wait	TPS(tok/s)
无显式 Residency	2684 MiB/token	25694 ms	1.77
当前 Residency 配置	612 MiB/token	6630 ms	5.18

Dense Residency Reduces Weight Streaming

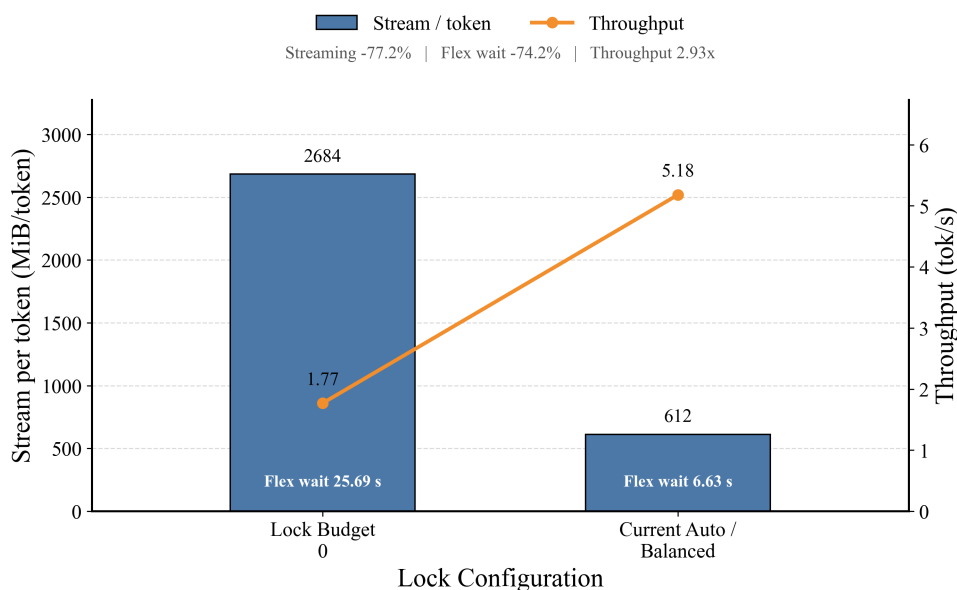


图 14

实验结果显示，引入显式权重 Residency 后，每 token 权重读取量由 2684 MiB 降低至 612 MiB，下降约 77.2%。同时，Flex Wait 从 25694ms 降低至 6630ms，吞吐由 1.77 tok/s 提升至 5.18 tok/s。

该结果说明，在 Dense 模型中，合理维护权重驻留集合能够显著减少重复 Weight Streaming，并降低同步 I/O 等待。

5.2.3 Risk-Aware Lock 避免内存风险边界

权重锁定策略决定了模型权重在内存中的常驻比例，直接影响每 token 的流式读取量与 OOM 风险。为评估低内存环境下的策略鲁棒性，本系统对比了三种锁定策略：

- Legacy Auto Proxy: 保守旧策略，倾向于维持比较小的锁定权重规模；
- Greedy Max Proxy: 贪心策略，追求最大化锁定以减少流式访问；
- Risk-Aware Auto v2: 风险感知自动策略，综合考虑内存限制、运行时开销与流式收益；

实验条件：

- memory.max = 2000MB
- ctx = 2048
- tokens = 128

- repeats = 3

实验结果如表 8 所示。

表 8: Risk-Aware Lock 策略测试结果

策略	Stream/token	Flex Wait	TPS	状态
Legacy Auto	1682 MiB	37539 ms	2.31	安全运行
Greedy Max	—	—	—	3/3 OOM
Risk-Aware Auto v2	1382 MiB	34193 ms	2.53	安全运行

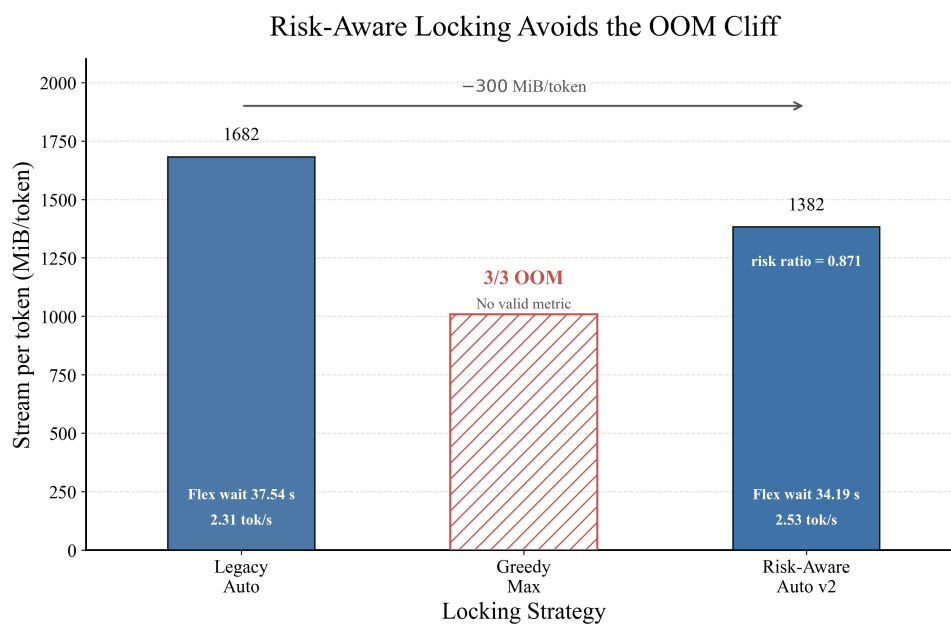


图 15

结果表明，简单增加锁定权重规模并不能持续提升性能。Greedy Max 策略由于忽略剩余内存压力，在三次实验中均触发 OOM。相比之下，Risk-Aware Auto v2 根据剩余内存和权重访问需求动态确定锁定规模，在保证可运行性的同时，将 Stream/token 从 1682 MiB 降低至 1382 MiB，并将吞吐提升至 2.53 tok/s。

该实验说明，在端侧低内存推理环境中，权重驻留策略需要同时考虑性能收益与内存风险，而非单纯追求最大化驻留。

5.2.4 Dense Pin Policy 对权重访问性能的影响

除决定权重驻留规模外，驻留对象的选择同样会影响 Dense 模型的 I/O 行为。由于不同 Tensor 在推理过程中的访问频率和计算贡献不同，如果仅按照大小或固定顺序选择驻留对象，可能导致有限内存无法覆盖关键访问路径。

因此，本系统设计多种 Pin Policy，包括：

- small-first: 优先驻留小尺寸 Tensor;
- large-first: 优先驻留大尺寸 Tensor;
- attn-first: 优先保护 Attention 相关权重;
- ffn-first: 优先保护 FFN 权重;
- cost-aware: 根据访问代价选择驻留对象;
- cost-aware-balanced: 结合访问频率和大小进行平衡;
- none: 无主动驻留策略，作为基线对照。

实验条件如下：

- memory.max = 3000MB
- ctx = 1024
- tokens = 64
- repeats = 3

完整测试结果如表 9 所示。

表 9: Dense Pin Policy 完整测试结果

策略	Flex Wait (ms)	TPS (tok/s)	说明
none	26448	1.80	无主动驻留，大量重复 I/O 等待
small-first	13386	2.88	优先驻留小张量，降低关键路径延迟
attn-first	14141	2.85	保护注意力权重，接近 small-first 效果
large-first	15969	2.62	优先驻留大张量，但覆盖关键层不足
cost-aware-balanced	17685	2.61	权衡频率与大小，但 overhead 略高
ffn-first	16851	2.57	保护 FFN 权重，收益低于 attention
cost-aware	18403	2.54	纯代价驱动，但在该条件下未达最优

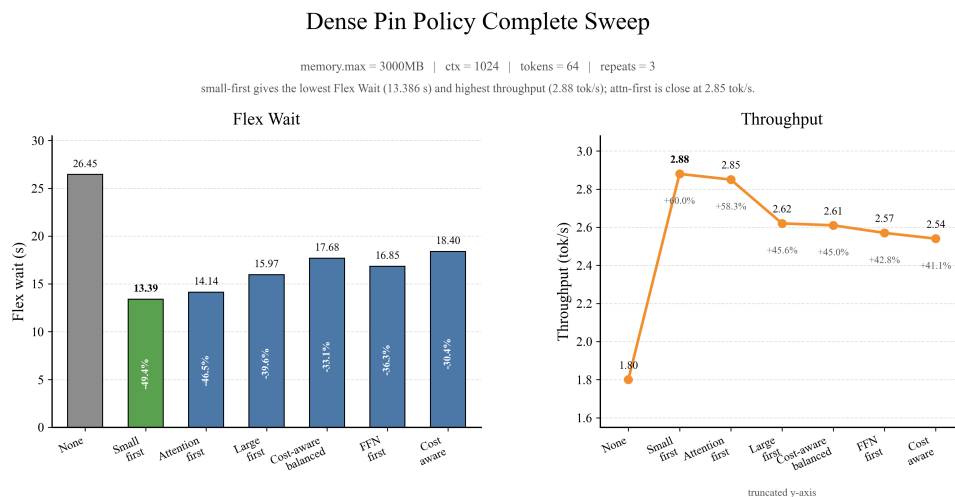


图 16

结果表明，在相同内存预算下，驻留对象的选择显著影响 Dense 模型的 I/O 等待和吞吐。相比无策略基线，所有主动策略均有效降低 Flex Wait，提升 TPS。其中 **small-first** 表现最佳，Flex Wait 从 26448 ms 降至 13386 ms，吞吐提升 60% 至 2.88 tok/s; **attn-first** 紧随其后，说明 Attention 层权重对性能至关重要。**large-first** 和

`ffn-first` 虽有改善, 但不如前两者, 表明简单按大小或单一模块选择并非最优。基于代价的 `cost-aware` 和 `cost-aware-balanced` 在本实验条件下反而略逊于 `small-first`, 可能因其计算开销或代价估计偏差削弱了收益。

综上, 权重驻留不仅是容量分配问题, 更涉及驻留对象的智能选择。`small-first` 和 `attn-first` 在该内存预算下提供了较好的性价比, 后续可结合代价模型进一步优化以适配更广泛的负载场景。

5.2.5 Adaptive Ahead 权重预取调度

在 Dense 模型中, 即使存在有效权重驻留, 如果预取距离不足, 计算线程仍可能等待下一层权重加载完成。因此, 本系统进一步设计 Adaptive Ahead 调度机制, 根据当前 I/O 压力和计算窗口动态调整预取提前量, 避免固定 Ahead 参数导致的性能不稳定。作为对照, 同时测试了固定 `ahead0/1/2/4` 四种距离配置。

实验条件如下:

- `memory.max = 2400MB`
- `ctx = 2048`
- `tokens = 128`
- `repeats = 3`

测试结果如表 10 所示。

表 10: Adaptive Ahead 机制与固定 Ahead 策略对比

策略	Stream/token	Flex (ms)	Wait	TPS (tok/s)	说明
ahead0	1682 MiB/to- ken	36511		2.36	无有效前瞻预取， I/O Stall 明显
ahead1 fixed	1682 MiB/to- ken	36404		2.38	预取距离仍不足， 收益有限
ahead2 fixed	1682 MiB/to- ken	38501		2.33	固定小窗口未缓 解 I/O 等待
Adaptive Ahead	1036 MiB/to- ken	23129		3.32	动态调整预取窗 口，降低 Stream/- token
ahead4 fixed	1682 MiB/to- ken	377		4.94	固定大预取窗口， 几乎完全隐藏 I/O 延迟

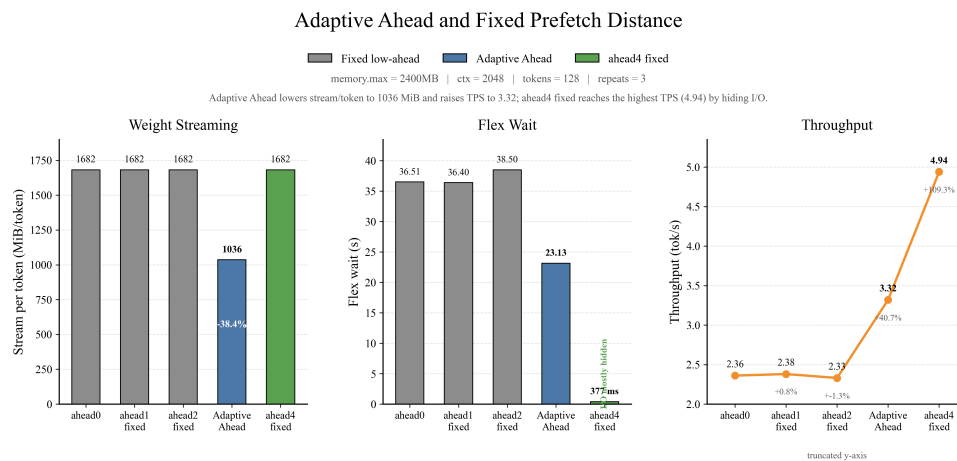


图 17

实验结果显示，Adaptive Ahead 通过动态调整预取窗口，有效避免了固定低 Ahead 参数造成的同步 I/O 等待，将每 token 权重流量由 1682 MiB 降低至 1036 MiB，Flex Wait 降低约 36%，TPS 提升至 3.32 tok/s。

值得注意的是，固定 ahead4 策略在当前测试中获得最高 TPS (4.94 tok/s)，但其记录到的 Stream/token 指标仍为 1682 MiB/token。二者优化路径不同：Adaptive Ahead 主要通过动态窗口降低本轮记录到的流式读取压力；而 ahead4 fixed 虽然 Stream/token 指标未降低，但较大的固定预取窗口显著隐藏了 I/O 延迟，因此 Flex Wait 降至 377 ms，

TPS 达到最高。这说明 Adaptive Ahead 当前策略更偏向安全调节，而非盲目扩大预取窗口。尽管动态策略尚未在所有 workload 下超越固定最优参数，但该结果证明了自适应调节机制能够有效改善低 Ahead 场景下的 I/O Stall，具备良好的鲁棒性和可扩展性。

5.3 MoE 模型权重管理测试结果

MoE 模型因稀疏专家激活机制，每个 token 仅激活部分 expert，其推理过程中的内存瓶颈不同于 Dense 模型——并非需要减少所有权重访问，而是需要在有限内存中维护高概率访问的专家工作集，并避免频繁的专家驱逐与重加载。本系统通过 Expert-level Buffer、状态机管理、Working-Set-Aware Budget Planner 以及 Pressure-Aware 自适应调度，实现专家权重的运行时管理。

实验主要从以下三个方面展开：

1. Expert Buffer 工作集边界与自动预算规划能力；
2. 压力感知自适应策略在不同内存预算下的稳定性；
3. 细粒度事件级诊断与预取准入机制分析。

5.3.1 MoE Buffer 工作集边界分析

首先测试不同 Expert Buffer budget 下系统运行状态。实验结果如表 11 所示。

表 11: MoE Buffer Working-Set 边界测试

budget(MB)	Evictions	Read/token	TPS(tok/s)	状态
768	7392	598	1.97	严重抖动
1536	2710	272	2.23	过渡状态
2048	618	132	6.22	接近收敛
2432	0	~0	11.57	工作集边界
2560	0	~0	9.43	过载状态

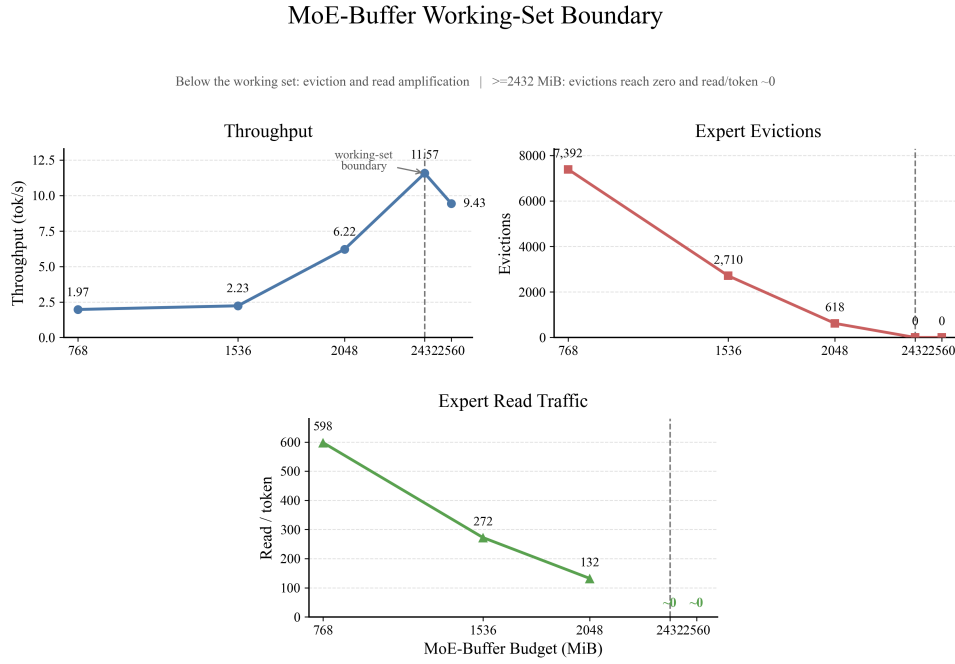


图 18

结果表明，MoE Buffer 存在明显的工作集边界。当 Buffer 小于活跃 expert 工作集规模时，系统需要不断执行 COLD $\rightarrow$ INFLIGHT $\rightarrow$ RESIDENT 状态转换，导致频繁 eviction 和 reload。因此在 768MB 和 1536MB budget 下，系统分别产生 7392 次和 2710 次 eviction，吞吐仅为 1.97 和 2.23 tok/s。当 budget 增大至 2432MB 后，eviction 降低为 0，read/token 接近 0，说明当前 workload 下 Buffer 已覆盖主要访问 expert 集合。

该实验说明，MoE 推理性能的关键并非无限扩大缓存，而是准确覆盖活跃专家工作集。

5.3.2 MoE Budget Planner 自动预算选择

固定 Expert Buffer 大小无法适应不同内存限制。因此，本系统设计 Working-Set-Aware Budget Planner，根据内存预算限制、运行时峰值内存、expert 工作集规模以及安全边际动态选择可运行预算。

在 1500MB memory hard cap 下测试结果如表 12 所示。

表 12: MoE Budget Planner v2 测试结果

策略	Expert Budget	OOM 次数	结果
Fixed 128MB	128MB	3/3	无法运行
Fixed 512MB	512MB	3/3	无法运行
Fixed 768MB	768MB	3/3	无法运行
Fixed 1024MB	1024MB	3/3	无法运行
Auto Planner v2	836MB	0/3	稳定运行

实验结果显示，在 1500MB 严格内存限制下，所有固定预算策略均无法稳定运行，而 Auto Planner v2 根据当前运行状态选择 836MB Expert Buffer，实现三次实验全部成功。

该结果证明，MoE Expert Buffer 的核心问题不是简单设置缓存大小，而是需要根据运行时工作集和内存约束动态规划可行预算。

5.3.3 Pressure-Aware Full Adaptive 跨内存预算测试

进一步测试 Pressure-Aware Full Adaptive 在不同 memory cap 下的稳定性。实验结果如表 13 所示。

表 13: Pressure-Aware Full Adaptive 跨内存预算测试

memory cap	相对 Adaptive-off TPS 变化	状态	说明
1500MB	+1.51%	提升	低内存压力
1800MB	+1.64%	提升	稳定收益
2000MB	+0.99%	提升	中等压力
2400MB	+0.51%	提升	高预算场景

Pressure-Aware Full Adaptive Across Memory Budgets

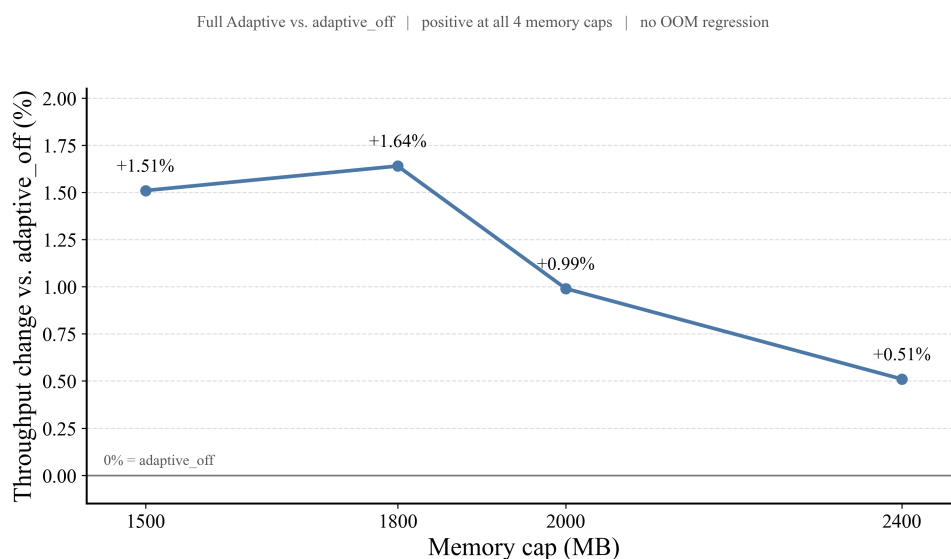


图 19

结果显示，Pressure-Aware Full Adaptive 在四档 memory cap 下均保持正向收益。相比固定策略，其优势并非某一个单点达到最高吞吐，而是在不同压力等级下均能够维持稳定运行。

该实验说明，MoE 调度需要根据实时内存压力动态调整保护范围，而不是采用固定窗口或固定驻留策略。

5.3.4 MoE 事件级调度诊断

为了进一步分析 MoE Buffer 在低内存场景下的运行行为，本文引入 event-level trace，对 Expert Buffer 生命周期中的 eviction、reload、read amplification 等事件进行统计。相比仅观察最终吞吐，事件级分析能够定位缓存策略退化的具体原因。

测试条件如下：

- memory.max = 2400MB
- ctx = 2048
- tokens = 256
- repeats = 1

实验结果如表 14 所示。

表 14: MoE Event Trace 诊断结果

指标	Current Policy	Active Window Off	说明
Peak RSS	2399.9 MiB	1721 MiB	内存压力接近限制
Evictions	16290	1962	大量专家驱逐
Read Bytes	18.44 GB	3.14 GB	I/O 放大明显
Cache Hit	0.9539	0.9927	命中率下降
TPS	7.07	8.60	性能下降

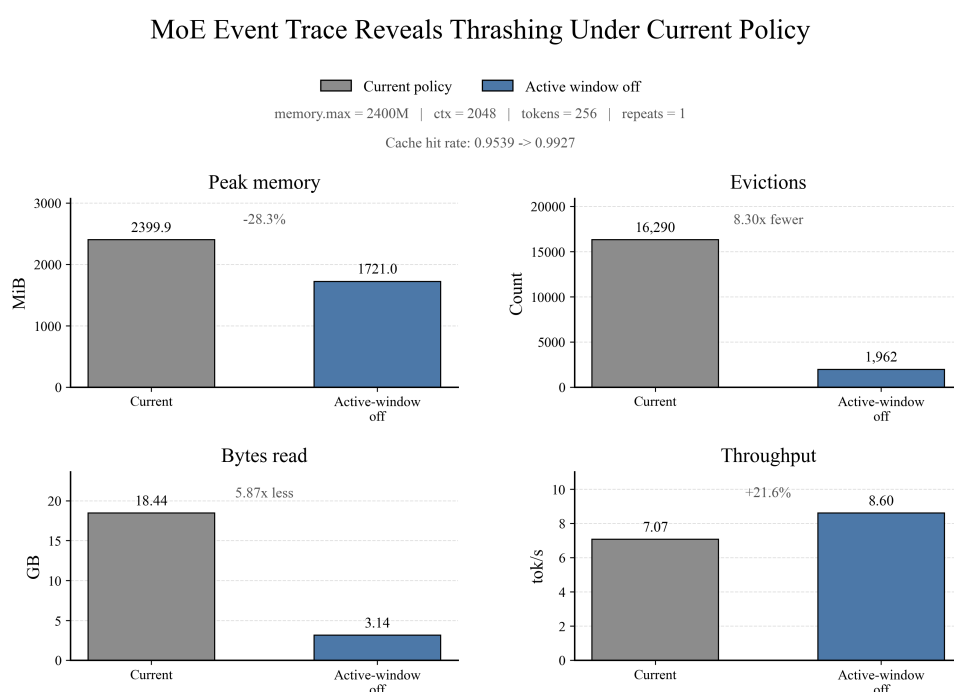


图 20

结果显示，在当前策略下，Expert Buffer 长时间维持较高驻留压力，导致系统不断触发 Resident Pressure → Eviction → Reload → Read Amplification 过程。Current Policy 相比 Active Window Off: eviction 次数增加约 8.3 倍，文件读取量增加约 5.9 倍，cache hit 从 0.9927 降低至 0.9539。

该结果说明，在部分低内存 workload 下，过强的 resident 保护可能反而导致缓存抖动。事件级 trace 为后续 Pressure-Aware 调节和动态预算分配提供了直接优化依据。

5.4 KV Cache 管理模块测试结果

KV Cache 部分围绕“物理驻留控制、恢复关键路径和多会话生命周期”三个方面开展测试。容量侧使用 mincore 与 KV object resident view 观测 KV tensor 的真实物理驻留页，并分别统计 RELEASE 与 OFFLOAD 带来的物理释放量；恢复侧记录 grouped backing read、prefault、scatter、restore wall time 与 graph gate；多会话侧通过 trace-driven replay 验证 Session 在 ACTIVE/IDLE/REVISIT/DEAD 状态间的完整运行过程。

受控容量实验使用 Qwen1.5-MoE-A2.7B Q4_K_M 模型、CPU 推理和单 slot 长上下文 workload。容量控制实验采用 F32 K/V，后续 KV 精度与多会话实验采用 F16 K/V。不同 KV 精度仅在相同实验条件下进行直接 A/B 比较，以保证各项数据具有清晰的归因关系。

5.4.1 Physical Resident Budget 控制曲线

在固定模型、workload 与 binary 的 KV Resident Budget sweep 中，以全驻留 Resident 作为基线，并逐步收紧 steady KV tensor resident target。实验矩阵覆盖 8 个 case、每个 case 2 轮，共 16 次运行，各预算点均能够稳定达到设定的 KV resident target。结果如表 15 所示。

表 15: KV Resident Budget 受控实验结果

策略 / target	实际 KV resident	相比 Resident 节省	steady TPOT 变化	resume gate
Resident	约 3.000 GiB	0	baseline	0
RELEASE 2.5 GiB	2.495 GiB	16.8%	+6.6%	—
RELEASE 2.0 GiB	1.995 GiB	33.5%	+6.0%	—
RELEASE 1.5 GiB	1.496 GiB	50.1%	+3.1%	—
V2 1.0 GiB	0.996 GiB	66.8%	+4.8%	约 190 ms
V2 0.75 GiB	0.746 GiB	75.1%	+7.9%	约 277 ms
V2 0.50 GiB	0.497 GiB	83.4%	+5.1%	约 370 ms
V2 0.25 GiB	0.247 GiB	91.8%	+4.2%	约 478 ms

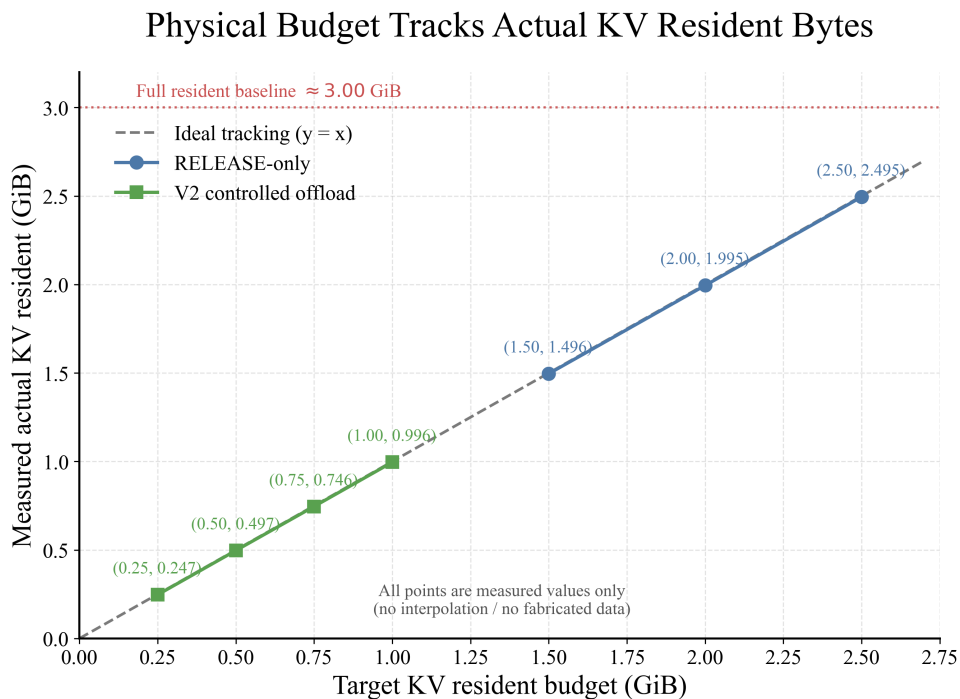


图 21

从容量侧看, Physical Resident Budget Controller 能够将 steady KV physical resident 从约 3.00 GiB 连续控制到 0.247 GiB, 对应最大约 91.8% 的 KV 物理驻留压缩。从恢复侧看, resident target 越低, 被迁移到 backing store 的 live KV 越多, Session 再次访问时需要恢复的数据量和一次性 resume gate 随之增加: 在 1.00 GiB、0.75 GiB、0.50 GiB 和 0.25 GiB 四个预算点, resume gate 分别约为 190 ms、277 ms、370 ms 和 478 ms。

与此同时, 完成恢复后的 steady TPOT 在该组实验中保持在 Resident 基线约 +3% 至 +8% 的范围内, 且没有随 resident target 呈单调恶化。该结果说明, KV 内存优化的主要权衡集中在“物理驻留量”与“Session 恢复代价”之间, 因此 FlexKV-OS 将 resident budget 控制与 Exact Restore 路径作为两个相互配合的优化层。

5.4.2 RELEASE 与 OFFLOAD 的物理收益归因

FlexKV-OS 根据 KV 的生命周期状态采用两级容量管理机制: 对于已经失去 Session 所有权、无需再次恢复的 dead/unowned KV, 优先执行 RELEASE; 对于仍可能被后续请求再次访问的 idle live KV, 则通过 OFFLOAD 将数据迁移到有界 backing store。两类机制对应不同的数据生命周期, 其物理内存收益分别统计, 如表 16 所示。

表 16: RELEASE / OFFLOAD 物理驻留收益

阶段		KV physical resident / relief	相比前一阶段	证据口径
Resident B_{full}		3,221,028,864 B (约 3.00 GiB)	—	KV physical resident
RELEASE	floor	1,386,479,616 B (约 1.291 GiB)	释放 1.709 GiB (56.96%)	KV physical resident
OFFLOAD	额外 relief	1,286,012,928 B (约 1.198 GiB)	额外真实物理下降	transaction-local mincore

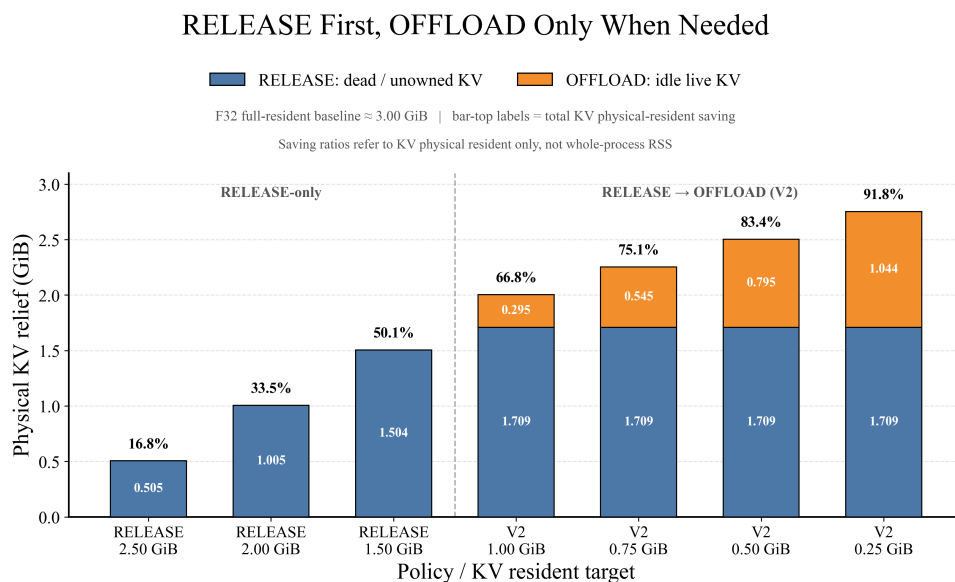


图 22

RELEASE-only 可将 KV physical resident 从约 3.00 GiB 降至 1.291 GiB，释放约 1.709 GiB (56.96%)，且过程不产生 swap-out、swap-in、backing write、backing read 或 positive restore，因此适合作为系统的第一层低开销容量回收手段。在此基础上，OFFLOAD 对 idle live KV 进一步产生约 1.198 GiB 的 transaction-local physical relief，使系统能够在保留 Session 语义的同时继续降低 DRAM 中的 KV 驻留量。

两类机制分别对应“无需保留内容的数据”和“需要保留语义但可以暂时移出 DRAM 的数据”，共同构成 FlexKV-OS 的 RELEASE-first、OFFLOAD-on-demand 分层内存管理路径。系统同时独立记录 logical KV bytes、backing I/O bytes 与 physical relief，从而分别刻画逻辑数据规模、数据迁移成本和真实物理内存收益。

5.4.3 Exact Restore 执行器消融

对于 OFFLOAD 到 backing store 的 KV，Session 再次访问前需要完成 Exact Restore。FlexKV-OS 将恢复路径进一步拆分为 Transfer Group、Read/Restore Pipeline 和 Destination Prefault 三个执行层，以降低大量 KV 数据重新进入物理内存时的阻塞开销。

Transfer Group 首先将相邻 SWAPPED block 合并为连续传输单元。在 1024-token workload 中，16 个 SWAPPED block 被组织为 8 个 transfer group，384 MiB Exact payload 由逐 block 读取收敛为每 group 一次连续 range read。同一实验中，mincore 观测到 399,507,456 B (381 MiB) 的真实 KV resident drop。384 MiB 表示 backing I/O payload，381 MiB 表示实际物理驻留变化，两者分别用于描述数据搬运量和 DRAM 释放量。

在 Transfer Group 基础上，Read/Restore Pipeline 使用固定 lookahead=1 的两级流水：

$$\text{READ}(G_{i+1}) \parallel \text{RESTORE}(G_i)$$

实验运行中，首个 group 完成读取后，后续 7 个 group 的 backing read 均能够与前一个 group 的 restore 过程重叠，记录到 `exposed_read_wait_us=0`、`pipeline_stall_us=0`。流水执行期间最多同时保留两个 task-owned staging group，峰值为 100,663,296 B (约 96 MiB)，因此额外工作集保持有界。

Destination Prefault 则在 tensor scatter 前使用 `MADV_POPULATE_WRITE` 预先建立目标物理页。三轮交错 OFF/ON A/B 中，6 次运行均保持 HTTP 200、response byte-exact、381 MiB physical resident relief 与 Exact graph gate 正常。关键计时如表 17 所示。

表 17: R2 Exact Restore Prefault 三轮中位数

指标	Prefault OFF	Prefault ON	变化
k2_pipeline_wall_us	237,277	200,885	-15.34%
restore_scatter_us	179,736	57,786	-67.85%
restore_prefault_us	0	84,886	+84,886 us
derived	179,736	142,672	-20.62%
physical_restore_us			
scatter minor faults	97,536	0	全部前移
prefault minor faults	0	97,536	全部前移
major faults	0	0	不变

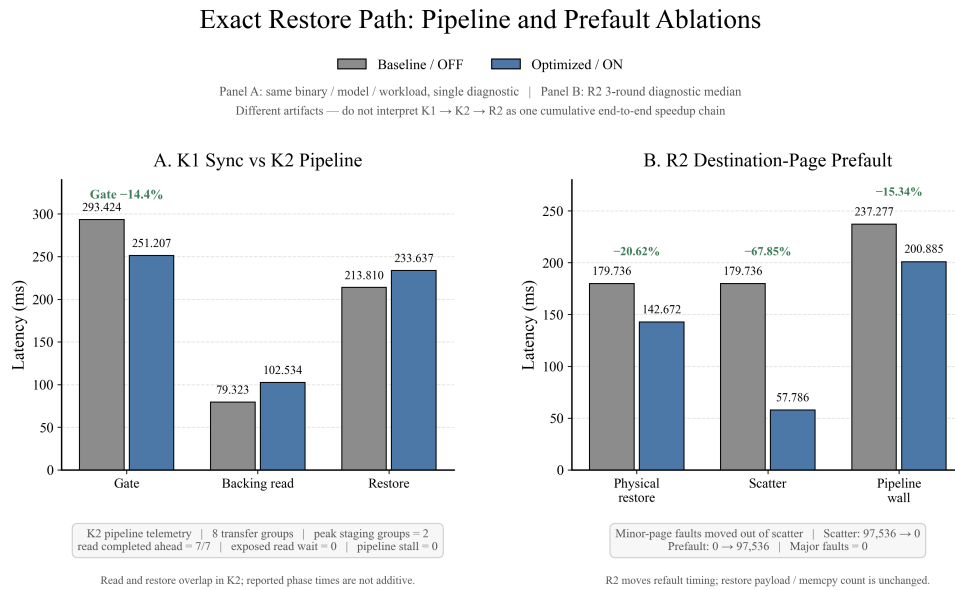


图 23

381 MiB 的 KV 物理页对应

$$381 \text{ MiB} / 4 \text{ KiB} = 97,536$$

个 4 KiB 页面。Prefault 开启后，97,536 个 minor page fault 从 tensor scatter 阶段整体前移到 prefault 阶段，使 scatter wall time 从 179.736 ms 降至 57.786 ms；计入 prefault 本身开销后，physical_restore_us 三轮中位数由 179.736 ms 降至 142.672 ms，降低 20.62%。这一结果表明，Exact Restore 的开销不仅来自 backing read，还包括目标页重新建立和 tensor 数据写回，因此系统通过 grouped I/O、流水重叠与物理页预建立共同优化恢复关键路径。

5.4.4 F16 KV 精度与恢复数据量

KV 元素精度直接决定 resident footprint 和 OFFLOAD/restore 的数据搬运量。FlexKV-OS 支持 native F16 paged KV，并在近似相同的相对 resident 压力下与 F32 进行对照，以隔离 bytes-per-KV 对物理内存和恢复路径的影响。结果如表 18 所示。

表 18: 近似等相对压力下 F32/F16 KV 对照

指标	F32	F16	变化
full-KV physical resident	约 3.00 GiB	约 1.50 GiB	约减半
恢复数据量	840 MiB	465 MiB	-44.64%
resume gate	471.318 ms	259.381 ms	-44.97%

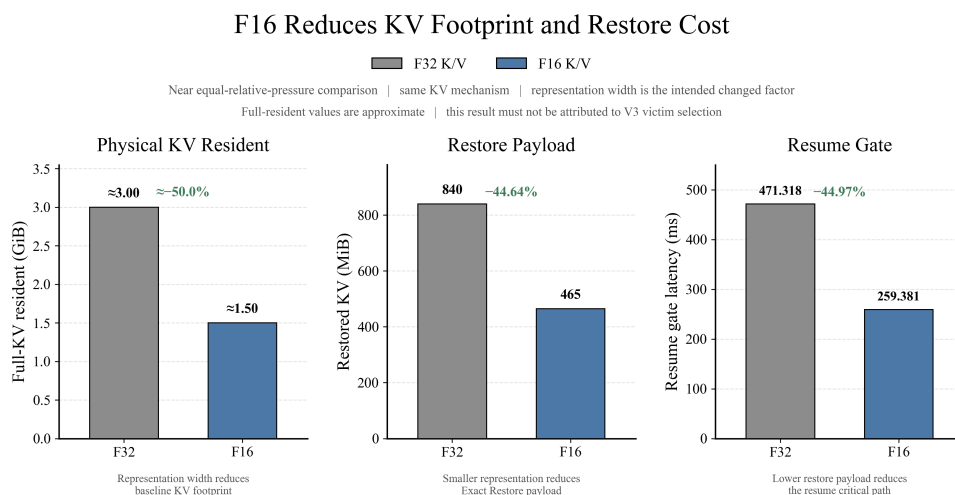


图 24

在相近的相对 resident 压力下，F16 将 full-KV physical resident 从约 3.00 GiB 降至约 1.50 GiB；对应恢复数据量由 840 MiB 降至 465 MiB，resume gate 由 471.318 ms 降至 259.381 ms。恢复数据量与 gate latency 的变化幅度接近，说明 Exact Restore 的端到端成本与实际需要搬运和重新驻留的 KV 字节量具有较强相关性。

F16 gather 采用 type-aware 的 `ggml_get_rows` 路径，并包含 F16→F32 的计算转换，因此该路径同时兼顾 KV 存储密度与现有计算图的数据类型要求。

5.4.5 真实多会话 workload 执行

除受控单 Session workload 外，系统还建立了 Alibaba usage trace 到真实 llama-server 的多会话执行链：

Frozen Trace → Model-bound Tokenization
→ Multi-session Slot Binding
→ Completion-driven Lifecycle

Trace 中的逻辑 Session 被稳定映射到独立 slot，并在真实 server 上按照请求到达顺序执行。运行结果如表 19 所示。

表 19: 真实多会话 KV workload 执行结果

验证层	真实运行结果	系统作用
Model-bound transcript	real tokenizer, direct_token_ids, effective n_ctx=8192	将 trace 请求与实际模型 token 及上下文配置绑定。
Multi-session replay	2 个 logical session 稳定绑定不同 slot; HTTP 200; zero loss/duplicate; event order PASS	保持 Session 独立生命周期与真实 arrival/revisit 顺序。
Completion lifecycle	3 个完成 turn; revisit_count=1、 ttl_expiry_count=2、 dead_count=2	驱动 ACTIVE/IDLE/REVISIT/TTL/DEAD/COLD_RESTART 状态转换。

该执行链使 KV Runtime 能够在多 Session 条件下观察真实的请求到达、空闲、再次访问和生命周期结束过程，并将这些事件统一映射为 Governor 可使用的 claimant 状态。与单 Session 容量实验相比，多会话 replay 为 idle age、reuse history、resident lease 和 churn 等运行时特征提供了真实的 Session 时间关系，也为 Hot/Cold 策略提供统一的 workload 输入。

5.5 全局自动调度下的内存容量边界与性能

在前述 Dense 权重驻留、MoE 专家工作集和 KV Cache 管理模块独立优化的基础上，本系统进一步将 Weight Residency / Flex Streaming、KV Cache Reclaim 以及 MoE Budget Planner 等机制统一纳入 Global Memory Governor 框架。本节通过容量边界实验，评估该框架在极端内存限制下的可运行性及性能表现。

实验采用固定 workload (ctx=2048, np=1, reqs=1, tokens=64)，在 7 个 cgroup memory.high 压力点下分别测试 Dense 与 MoE 模型。所有组合共 56 个 runs，每个条件仅执行 1 次重复 (REPEATS=1)，作为容量边界筛选实验。

实验主要从以下三个方面展开：

1. 不同全局策略下的最低可运行内存门槛；
2. 成功运行时各策略的吞吐对比；
3. 内存占用与性能之间的权衡关系。

5.5.1 容量边界与可运行性

首先统计各策略在不同内存压力点下的启动成功率。结果如表 20 所示。

表 20: 不同模型与全局策略在 7 个内存压力点下的成功运行数（共 7 点）

模型	baseline	kv_only	weight_only	combined_auto
Dense	2/7	2/7	3/7	2/7
MoE	1/7	1/7	6/7	6/7

进一步细化各策略的最低成功内存点，如表 21 所示。

表 21: 各策略最低可运行内存点（memory.high）

模型	策略	最低成功 memory.high
Dense	baseline	2300M
	kv_only	2300M
	weight_only	2000M
	combined_auto	2300M
MoE	baseline	2800M
	kv_only	2800M
	weight_only	1300M
	combined_auto	1300M

实验结果表明：

- **Dense 模型**的容量边界受限于完整模型权重驻留需求，即使启用权重优化，最低可运行内存门槛仍为 2000M（weight_only），且 combined_auto 未能进一步降低门槛，说明当前 Dense 侧低内存瓶颈主要来自权重本身的访问模式，而非 KV 缓存。
- **MoE 模型**则表现出显著差异：baseline 与 kv_only 仅在最高压力点（2800M）才能成功启动，而 weight_only 与 combined_auto 可将可运行门槛直接降至 1300M，成功率达 6/7。这证明了 Expert Buffer 与 Budget Planner 在低内存环境下的关键作用——它们能将 MoE 推理从“全量加载”模式转变为“动态工作集”模式。

5.5.2 成功运行点的性能对比

在可运行的条件下，进一步对比各策略的生成吞吐（tok/s）。

Dense 成功点性能如表 22 所示。

表 22: Dense 模型成功运行点吞吐对比

memory.high	baseline	kv_only	weight_only	combined_auto
2000M	—	—	1.87	—
2300M	1.72	2.64	2.40	2.94
2800M	3.09	13.45	2.42	4.73

在 Dense 侧：

- 在 2300M 点，combined_auto 吞吐最高（2.94 tok/s），优于 baseline（1.72）和 kv_only（2.64），说明联合调度在该压力下能更有效地平衡权重与 KV 内存。
- 在 2800M 点，kv_only 吞吐异常高（13.45 tok/s），这源于该条件下 KV 回收释放了大量内存，使系统能以接近全速运行；而 combined_auto 此时仍保留部分权重优化开销，吞吐略低（4.73），但依然显著高于 baseline（3.09）。

MoE 模型成功点性能如表 23 所示。

表 23: MoE 模型成功运行点吞吐对比

memory.high	baseline	kv_only	weight_only	combined_auto
1300M	—	—	1.37	1.12
1600M	—	—	2.38	6.26
1800M	—	—	8.51	8.28
2000M	—	—	8.65	9.26
2300M	—	—	8.97	11.10
2800M	7.34	17.60	8.85	11.46

MoE 侧的关键发现：

- weight_only 和 combined_auto 从 1300M 起即可运行，而 baseline 和 kv_only 直到 2800M 才成功。这表明 MoE 的低内存能力完全由权重管理决定，KV 回收在极低内存下无法单独发挥作用。

- `combined_auto` 在 1600M、2000M、2300M 和 2800M 均优于 `weight_only`，尤其在 1600M 处吞吐差距明显（6.26 vs 2.38），说明联合调度在中低内存下能更智能地分配资源，释放更多空间给专家工作集。
- 在 1300M 和 1800M 处，`combined_auto` 略低于 `weight_only`，这可能源于联合调度的保守策略或单次运行的波动。
- 在 2800M 点，`kv_only` 达到最高吞吐（17.60），但该策略无法在更低内存下运行；而 `combined_auto` 虽然吞吐较低（11.46），但能在所有低内存点保持可用，体现了鲁棒性优势。

5.5.3 内存-吞吐权衡分析

进一步观察成功运行时的峰值 RSS，如表 24 所示（选取代表性点）。

表 24: 关键内存点的峰值 RSS (MB) 与吞吐 (tok/s) 对比

模型	memory.high	策略	峰值 RSS / 吞吐
Dense	2800M	baseline	2802 MB / 3.09
		kv_only	2800 MB / 13.45
		weight_only	1921 MB / 2.42
		combined_auto	2598 MB / 4.73
MoE	2800M	baseline	2801 MB / 7.34
		kv_only	2800 MB / 17.60
		weight_only	1222 MB / 8.85
		combined_auto	1955 MB / 11.46
MoE	2300M	weight_only	1222 MB / 8.97
		combined_auto	1643 MB / 11.10

从内存-吞吐权衡可以看出：

- **MoE weight_only** 在所有成功点上均将 RSS 压至约 1.2GB，但吞吐随内存增加提升有限（从 1.37 升至 8.85），说明该策略在低内存下主要保证可运行性，对性能的优化空间受限于严格的驻留限制。
- **MoE combined_auto** 在同样条件下占用更多内存（如 2300M 点 1.64GB vs 1.22GB），但吞吐提升明显（11.10 vs 8.97），体现了“以内存换性能”的调度策略，且该策略在绝大多数内存点上优于 `weight_only`。

- **Dense 模型**中 `kv_only` 在 2800M 下取得极高吞吐，但其 RSS 也几乎用满限制，且该策略无法在更低内存下运行；`combined_auto` 在 2300M 和 2800M 均能保持比 `baseline` 更高的吞吐，同时占用内存略低于限制，说明它具备一定的安全边际。

5.6 系统测试结果总结

综合上述实验结果，本文提出的内存管理系统能够在 Dense、MoE 以及长上下文 KV 场景下实现统一的运行时资源管理。

主要实验结论如下：

1. 权重物理驻留管理能够显著降低基础内存占用。

Repack 优化实验表明，消除冗余权重副本后，Dense 和 MoE 模型 RSS 分别降低约 3.21GB 和 2.13GB。

2. Dense 模型需要结合 Residency、Budget 和 I/O 调度共同优化。

Window/Flex Buffer 能够降低重复权重加载，Risk-Aware Lock 能避免低内存条件下 OOM，Pin Policy 和 Adaptive Ahead 则进一步优化有限驻留空间中的访问效率。

3. MoE 模型优化核心在于维护有效 Expert 工作集。

实验发现，Expert Buffer 存在明显 Working-Set Boundary。低于该边界时会产生 eviction/reload 抖动，达到工作集规模后即可进入稳定命中状态。

4. 动态预算规划能够提升低内存环境适应能力。

MoE Budget Planner v2 在 1500MB hard cap 下，使固定预算全部失败的场景转变为稳定运行。

5. KV Cache reclaim 能够释放真实物理页。

mincore 实验表明，ctx4096 和 ctx8192 场景下，KV resident page drop 与 RSS drop 高度一致，证明系统释放的是实际物理页。

6. 权重与 KV 优化具有互补性。

联合实验显示，Weight Residency 与 KV reclaim 可以叠加降低 RSS，同时保持接近原始推理吞吐。

总体来看，本系统并非通过单一缓存策略降低内存，而是针对大模型推理过程中的不同内存来源，分别设计：

- 权重驻留控制；
- Expert 工作集管理；

- KV 生命周期管理;
- 压力感知调度。

通过统一 Memory Governor 进行协调, 系统能够在端侧有限内存条件下, 实现大模型推理过程中的动态资源分配, 并在降低 RSS 的同时保持可接受的推理性能。

6 当前系统不足与未来方向

尽管本系统已经完成核心功能验证, 但当前仍存在若干限制和后续优化空间。

6.1 运行时资源决策的自适应能力仍需进一步提升

当前系统已经具备基于运行时状态进行动态资源调整的能力, 并能够根据内存压力、数据访问特征和实际执行反馈持续修正资源分配策略。但现阶段实验主要集中于有限的模型类型、上下文规模和并发负载组合, 对于突发请求、长时间多会话运行以及不同访问模式频繁切换等复杂场景, 系统调度策略的稳定性和长期收敛性仍缺乏充分验证。未来可进一步扩大动态负载和长期运行测试范围, 重点分析频繁资源调整是否会引入额外抖动、策略切换开销或局部性能波动, 并进一步提高系统在复杂运行环境下的稳定性和鲁棒性。

6.2 极端内存约束下仍存在内存占用与推理性能的权衡

实验结果表明, 通过主动控制数据驻留和按需加载能够显著降低大语言模型推理所需的物理内存, 并使部分原本无法运行的低内存配置具备推理能力。但当可用内存持续减小时, 更多模型数据和运行时状态需要在内存与外部存储之间频繁迁移, 系统瓶颈会逐渐由内存容量转化为存储 I/O, 进而增加推理等待时间。因此, 未来优化不应单纯追求最低物理内存占用, 而应进一步研究内存占用、数据迁移开销与推理性能之间的动态平衡, 根据实际资源条件选择更合适的运行点。

6.3 系统泛化能力与实验验证范围仍有进一步扩展空间

当前系统已经在典型 Dense 模型、MoE 模型、长上下文以及多会话场景下完成验证, 但实验仍主要集中于有限模型规模、CPU 推理路径和当前存储环境。不同模型结构、量化格式、上下文长度及硬件平台可能具有不同的访存特征和最优资源配置。未来可进一步扩展至更大规模模型、更多端侧设备和不同存储带宽环境, 并探索 CPU、GPU、NPU 等异构设备下的统一运行时内存管理, 从而验证并提升系统在不同模型和硬件条件下的通用性与可扩展性。

7 大语言模型使用说明

项目开发过程中，我们合理使用了大语言模型作为辅助工具，具体使用方式如下：

1. **文献检索与资料整理：**借助开源大语言模型（如 DeepSeek V4 等）对相关研究论文进行语义检索与摘要生成，辅助快速定位技术路线和关键参考文献，提高调研效率。
2. **开源框架梳理与分析：**利用 LLM 对 llama.cpp、vLLM、SGLang 等开源推理框架的代码仓库结构、文档说明和实现机制进行快速梳理与对比分析，辅助理解各框架的权重管理、KV Cache 组织方式以及相关优化技术，为系统设计与方案选型提供参考。
3. **代码开发辅助：**在系统实现过程中，使用 LLM 辅助生成代码框架、编写测试脚本、调试错误及优化代码结构，所有生成代码均经人工审阅、修改和验证，确保符合项目需求和正确性。

需要特别说明的是，所有技术方案设计、核心算法实现、实验设计、数据分析与结论均完全由本项目成员独立完成，LLM 仅作为辅助工具使用，项目最终成果的完整性与真实性由本项目成员全权负责。

参考文献

- [1] Zixuan Zhou, Xuefei Ning, Ke Hong, et al., A Survey on Efficient Inference for Large Language Models, arXiv preprint, 2024.
- [2] Hongchao Du, Shangyu Wu, Arina Kharlamova, et al., FlexInfer: Breaking Memory Constraint via Flexible and Efficient Offloading for On-Device LLM Inference, arXiv preprint, 2024.
- [3] Keivan Alizadeh, Iman Mirzadeh, Dmitry Belenko, et al., LLM in a flash: Efficient Large Language Model Inference with Limited Memory, arXiv preprint, 2024.
- [4] Zhenliang Xue, Yixin Song, Zeyu Mi, et al., PowerInfer-2: Fast Large Language Model Inference on a Smartphone, arXiv preprint, 2024.
- [5] Zhiyuan Fang, Zicong Hong, Yuegui Huang, et al., Fate: Fast Edge Inference of Mixture-of-Experts Models via Cross-Layer Gate, arXiv preprint, 2024.
- [6] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, et al., Efficient Memory Management for Large Language Model Serving with PagedAttention, in *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023, pp. 611–626.

- [7] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, et al., SGLang: Efficient Execution of Structured Language Model Programs, arXiv preprint, 2024.
- [8] Yuhang Li, Rong Gu, Chengying Huan, et al., HotPrefix: Hotness-Aware KV Cache Scheduling for Efficient Prefix Sharing in LLM Inference Systems, arXiv preprint, 2025.
- [9] Krishna Teja Chitty-Venkata, Jie Ye, Xian-He Sun, et al., PagedEviction: Structured Block-wise KV Cache Pruning for Efficient Large Language Model Inference, arXiv preprint, 2024.
- [10] Ramya Prabhu, Ajay Nayak, Jayashree Mohan, et al., vAttention: Dynamic Memory Management for Serving LLMs without PagedAttention, in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2025.